

JEU D'ECHECS QUANTIQUES

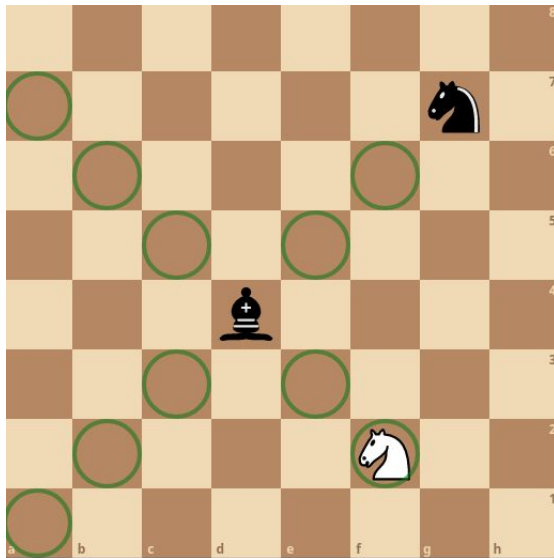
n°xxxxx Quentin Horgues



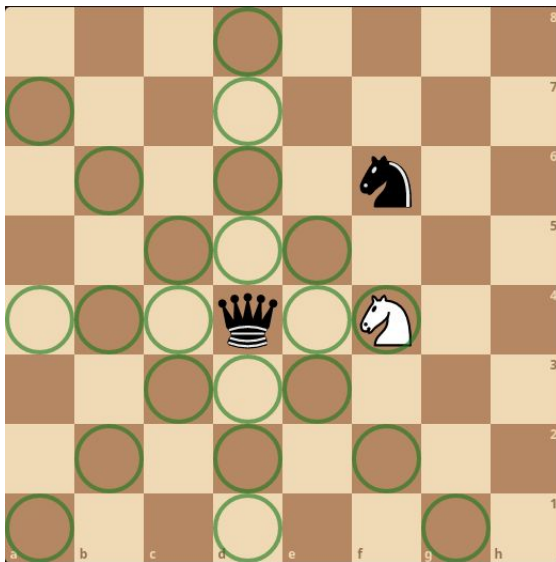
Présentation des règles du jeu

- 2 joueurs qui joue à tour de rôle
- Un tour = un mouvement
- Objectif : capturer le roi adverse

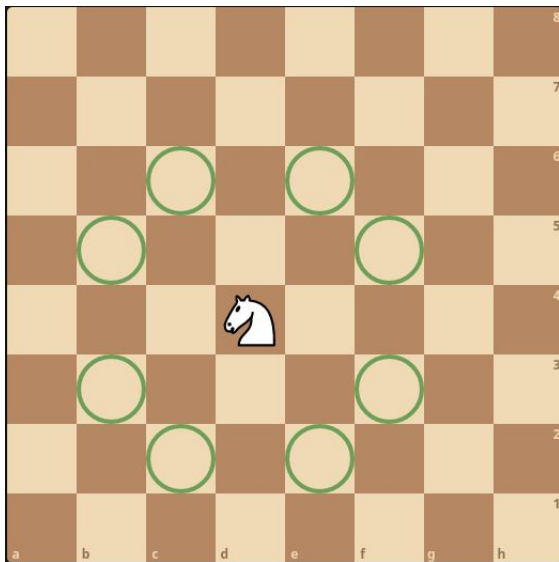
Mouvement classique : fou



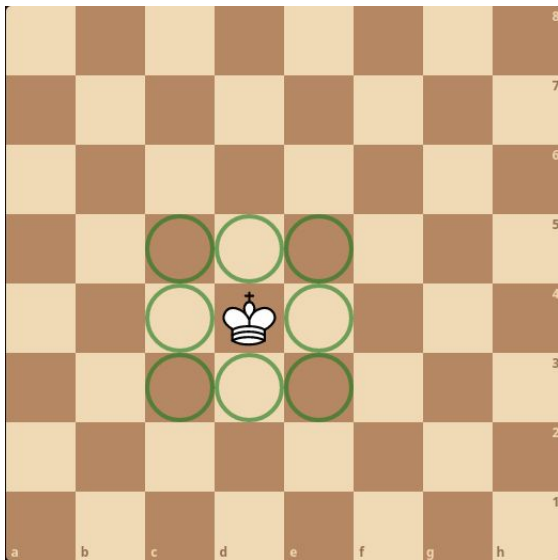
Mouvement classique : dame



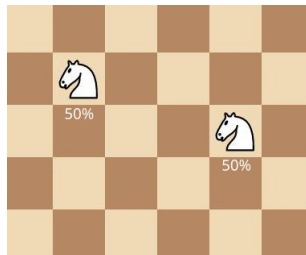
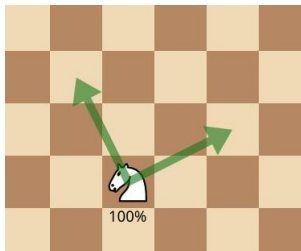
Mouvement classique : cavalier



Mouvement classique : roi



Mouvement split



```

#include <iostream>
#include <fstream>
#include <complex>
#include <string>
#include <Complex_printer.hpp>
#include <CMatrix.hpp>
#include <Unitary.hpp>
#include <Qubit.hpp>
#include <Board.hpp>
#include <Piece.hpp>
#include <Move.hpp>
#include <observer_ptr.hpp>
#include <check_path.hpp>
#include <Constexpr.hpp>
#include <ComputerPlayer.hpp>
#include <ConsoleInterface.hpp>

```

```

char piece_to_char(TypePiece piece, Color color = Color::WHITE)
{
    int const offset{(color == Color::BLACK) ? 32 : 0};
    using enum TypePiece;
    switch (piece)
    {
        case KING:
            return static_cast<char>('K' + offset);
        case QUEEN:
            return static_cast<char>('Q' + offset);
        case ROOK:
            return static_cast<char>('R' + offset);
        case BISHOP:

```



```

        return static_cast<char>('B' + offset);
    case KNIGHT:
        return static_cast<char>('N' + offset);
    case PAWN:
        return static_cast<char>('P' + offset);
    case EMPTY:
        return ' ';
    default:
        return '\\0';
    }
}

template <std::size_t N, std::size_t M>
char print_piece(Board<N, M> &board, Coord const &position)
{
    TypePiece piece{board(position.n, position.m).get_type()};
    Color color{board(position.n, position.m).get_color()};
    return piece_to_char(piece, color);
}

template <std::size_t N, std::size_t M>
void auto_playing(
    Board<N, M> &board,
    std::ostream &output,
    int nb_moves,
    int search_depth)
{
    std::cout << board << std::endl;
    while (
        nb_moves > 0 &&

```

```

        !board.winning_position(opponent_color(board.get_current_player()))
    }

    Move m{computer::get_best_move(board, search_depth)};

    std::string data1{{static_cast<char>('a' + m.normal.src.m)},
                                                                static
    std::string data2{{static_cast<char>('a' + m.normal.arv.m)},
                                                                static

    output << print_piece(board, m.normal.src) << ' ';
    if (m.type == TypeMove::NORMAL)
    {
        output << "N " << data1 << data2 << std::endl;
    }
    else if (m.type == TypeMove::PROMOTE)
    {
        output << 'P' << piece_to_char(m.promote.piece) << ' '
                << data1 << data2
                << std::endl;
    }
    else
    {
        std::string data3{{static_cast<char>('a' + m.split.arv2.m)},

    if (m.type == TypeMove::SPLIT)
    {
        output << "S " << data1 << '(' << data2 << ':'

```

```

        << data3 << ')' << std::endl;
    }
    else
    {
        output << "M " << '(' << data1 << ':' << data2 << ')'
            << data3 << std::endl;
    }
}
board.move(m);

/*
#ifdef WIN32
    int ret = system("cls");
#else
    int ret = system("clear");
#endif

(void)ret;*/
std::cout << board << std::endl;
board.change_player();
nb_moves--;
}

}

int main()
{
    Board<> const ChessBoard{
        {B_ROOK, B_KNIGHT, B_BISHOP, B_QUEEN, B_KING, B_BISHOP, B_KNIGHT, B_ROOK,
        {B_PAWN, B_PAWN, B_PAWN, B_PAWN, B_PAWN, B_PAWN, B_PAWN, B_PAWN},
        {Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(),
        {Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(),

```

```

{Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece()
{Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece(), Piece()
{W_PAWN, W_PAWN, W_PAWN, W_PAWN, W_PAWN, W_PAWN, W_PAWN, W_PAWN},
{W_ROOK, W_KNIGHT, W_BISHOP, W_QUEEN, W_KING, W_BISHOP, W_KNIGHT, W_RO

/* Board<3> B1{
    {B_KING, B_ROOK, Piece()},
    {B_PAWN, Piece(), Piece()},
    {Piece(), W_QUEEN, W_KING}};

Board<3> B2{
    {
        {B_KING, Piece(), B_QUEEN},
        {W_PAWN, B_PAWN, W_QUEEN},
        {Piece(), Piece(), W_KING}
    }
};

Board<3> B3{
    {
        {B_KING, Piece(), Piece()},
        {Piece(), W_ROOK, Piece()},
        {Piece(), Piece(), W_KING}
    }
};
*/

Board<6, 4> smallBoard{
    {{B_KNIGHT, B_QUEEN, B_KING, B_BISHOP},
     {B_PAWN, B_PAWN, B_PAWN, B_PAWN},

```

```

        {Piece(), Piece(), Piece(), Piece()},
        {Piece(), Piece(), Piece(), Piece()},
        {W_PAWN, W_PAWN, W_PAWN, W_PAWN},
        {W_KNIGHT, W_QUEEN, W_KING, W_BISHOP}}};

Board<4> miniBoard{
    {
        {B_KING, Piece(), Piece(), B_BISHOP},
        {Piece(), B_KNIGHT, Piece(), Piece()},
        {Piece(), Piece(), Piece(), Piece()},
        {Piece(), Piece(), W_QUEEN, W_KING}
    }
};

for (int i = 1; i <= 1; i++)
{
    Board board{ChessBoard};
    std::ofstream log_file{"partie" + std::to_string(i) + ".txt"};
    auto_playing(board, log_file, 80, 5);
    log_file.close();
}

/*for (std::size_t i = 0; i < 8; i++)
{
    for (std::size_t j = 0; j < 8; j++)
    {
        std::cout << computer::_utility::evalCase<8, 8>(i, j) << ' ';
    }
    std::cout << std::endl;
}*/

```

```
}    return 0;
```

```

#ifndef GAME_BOARD_HPP
#define GAME_BOARD_HPP

#include <vector>
#include <array>
#include <utility>
#include <optional>
#include <complex>
#include <cstdint>
#include <initializer_list>
#include <functional>
#include <CMatrix.hpp>
#include <Piece.hpp>
#include <TypePiece.hpp>
#include <Color.hpp>
#include <Coord.hpp>
#include <forward_list>
#include <observer_ptr.hpp>
#include <Move.hpp>
#include <Constexpr.hpp>
#include <check_path.hpp>

class Piece;

/**
 * @brief La classe représentant le plateau de jeu
 *
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonne du plateau
 */

```

```

template <std::size_t N = 8, std::size_t M = N>
class Board final
{
public:
    // Constructeur
    CONSTEXPR Board();
    /**
     * @brief Initialise le plateau à partir d'une liste de pièce,
     * les pièces ne sont pas divisées initialement.
     *
     * @param[in] board Double initializer_list sur des pointeurs sur Piece
     */
    CONSTEXPR Board(std::initializer_list<std::initializer_list<
        Piece*>> board);

    // Copie
    CONSTEXPR Board(Board const &) = default;
    CONSTEXPR Board &operator=(Board const &) = default;

    // Mouvement
    CONSTEXPR Board(Board &&) = default;
    CONSTEXPR Board &operator=(Board &&) = default;

    // Destructeur
    CONSTEXPR ~Board() = default;

    /**
     * @brief Retourne le nombre de ligne du plateau

```



```

*
* @return std::size_t Le nombre de ligne
*/
CONSTEXPR static std::size_t numberLines() noexcept;

/**
 * @brief Retourne le nombre de colonne
 *
 * @return std::size_t Le nombre de colonne
 */
CONSTEXPR static std::size_t numberColumns() noexcept;

/**
 * @brief Retourne un pointeur sur la piece à l'emplacement cible
 *
 * @param[in] n L'indice de la ligne
 * @param[in] m L'indice de la colonne
 * @return Un pointeur observateur sur une piece
 * ou Piece() si la case est vide
 */
CONSTEXPR Piece const &
operator()(std::size_t n,
           std::size_t m) const noexcept;

/**
 * @brief Renvoie la liste dans tous les mouvements
 * classiques légaux d'une pièce
 *
 * @param pos Position de la pièce
 */

```

```

* @return std::forward_list<Coord> La liste de coordonnées des positions
* d'arrivées de chaque mouvement possible
*/
CONSTEXPR std::forward_list<Coord>
get_list_normal_move(Coord const &pos) const;

/**
* @brief Renvoie la liste de toutes les cases qu'une pièce
* peut atteindre lors d'un mouvement split, il faut choisir
* deux éléments de la liste pour créer un mouvement split.
*
* @param[in] pos Position de la pièce.
*
* @return std::forward_list<Coord> La liste de coordonnées
* des positions d'arrivées possible sachant que chaque élément
* représente une seule des deux cases d'arrivées d'un split move
*/
CONSTEXPR std::forward_list<Coord>
get_list_split_move(Coord const &pos) const;

/**
* @brief Teste si les mouvement de la pièce sont si la
* pièce est un pion un mouvement de promotion
*
* @note peut être utiliser sans vérifier si la pièce est un pion
*
* @param[in] pos La position de la pièce
* @return true si le mouvement possible est un mouvement
* de promotion
* @return false sinon

```

```

*/
CONSTEXPR bool
check_if_use_move_promote(Coord const &pos) const noexcept;

/**
 * @brief Renvoie la liste de toutes les promotions
 *
 * @warning Ne verifie pas la validité du mouvement,
 * de la position ou du type de la pièce
 *
 * @param[in] pos La position du pion
 * @return La liste de tout les mouvements de promotion
 */
CONSTEXPR std::forward_list<Move>
get_list_promote(Coord const &pos) const noexcept;

/**
 * @brief Applique une fonction à l'entièreté des mouvements possible
 *
 * @param[in, out] func La fonction à appliquer sur tout les mouvements
 * Prototype : bool f(Move const&)
 * Si renvoie true alors le parcourt est interrompue
 * @param[in] color La couleur du joueur auquel on
 * récupère les mouvements
 */
template <class UnitaryFunction>
CONSTEXPR void
all_move(
    UnitaryFunction func,
    std::optional<Color> color_player = std::nullopt) const noexcept

```

```

/**
 * @brief Test si un mouvement est réalisable
 *
 * @param[in] move Le mouvement à tester
 * @return true si le mouvement est légal
 */
CONSTEXPR bool move_is_legal(Move const &move) const;

/**
 * @brief Réalise un mouvement d'une pièce quelque soit le type
 * du mouvement (classic, split, merge)
 *
 * @warning Ne procède aucune vérification sur la validité du mouvement
 *
 * @param[in] mouvement Le mouvement à réaliser
 * @param[in] val_mes Optional peut déterminer la mesure de
 * la présence d'une pièce ou non
 */
CONSTEXPR void
move(Move const &movement, std::optional<bool> val_mes = std::nullopt);

/**
 * @brief Test si un plateau est gagnant pour une couleur
 *
 * @param[in] c Couleur
 * @return true si la position est gagnante pour la couleur c
 * @return false sinon
 */
CONSTEXPR bool winning_position(Color c) const noexcept;

```

```

/**
 * @brief Recupère la couleur du joueur au tour de jouer
 *
 * @return Color la couleur du joueur
 */
CONSTEXPR Color get_current_player() const noexcept;

/**
 * @brief Change le joueur actuel et passe la main à l'autre joueur
 */
CONSTEXPR void change_player() noexcept;

/**
 * @brief Renvoie la probabilité qu'il y ait une pièce à une position.
 *
 * @param pos La position
 */
CONSTEXPR double get_proba(Coord const &pos) const noexcept;

friend class Piece;

template <std::size_t _N, std::size_t _M>
friend CONSTEXPR bool
check_path_straight_1_instance(
    Board<_N, _M> const &board,
    Coord const &dpt,
    Coord const &arv,
    std::size_t position,
    std::optional<Coord>);

```

```

template <std::size_t _N, std::size_t _M>
friend CONSTEXPR bool
check_path_diagonal_1_instance(
    Board<_N, _M> const &board,
    Coord const &dpt,
    Coord const &arv,
    std::size_t position,
    std::optional<Coord>);

```

```

template <std::size_t _N, std::size_t _M>
friend CONSTEXPR bool
check_path_queen_1_instance(
    Board<_N, _M> const &board,
    Coord const &dpt,
    Coord const &arv,
    std::size_t position,
    std::optional<Coord>);

```

```

/**

```

```

 * @brief Fonction qui permet de faire un mouvement de pion quelconque avec une
 *
 * @param s Coordonnées de la source
 * @param t Coordonnées de la cible
 * @param p Le type de la pièce de la promotion
 * @param[in] val_mes Optionel peut determiner la mesure de
 * la présence d'une pièce ou non
 */

```

```

CONSTEXPR void move_promotion(
    Move const &move,

```

```

std::optional<bool> val_mes = std::nullopt);

/**
 * @brief Renvoie la probabilité que le mouvement soit réalisé
 *
 * @param move Le mouvement à réaliser
 * @return Une probabilité comprise entre 0 et 1
 */
CONSTEXPR double get_proba_move(Move const &move) const noexcept;

```

private:

```

/**
 * @brief Renvoie la probabilité que la fonction mesure renvoie true,
 * c'est-à-dire la proba que le mouvement soit "réussi"
 *
 * @param p Coordonnées de la case
 * @return La probabilité
 */
CONSTEXPR double
get_proba_mesure(Coord const &p) const noexcept;

/**
 * @brief Renvoie la probabilité que la fonction mesure_capture_slide renvoie tr
 * c'est-à-dire la proba que le mouvement de capture_slide soit "réussi"
 *
 * @param s Coordonnées de la source
 * @param t Coordonnées de la cible
 * @param check_path Fonction qui teste la présence d'une pièce

```

```

* entre une source et une cible
* @return La probabilité
*/
CONSTEXPR double get_proba_mesure_capture_slide(
Coord const &s,
Coord const &t,
std::function<bool(Board<N, M> const &,

                                check_path) const noexcept;

/**
 * @brief Renvoie la probabilité que la fonction mesure_castle renvoie true,
 * c'est-à-dire la proba que le mouvement de roque soit "réussi"
 *
 * @param king Coordonnées du roi
 * @param rook Coordonnées de la tour
 * @return La probabilité
 */
CONSTEXPR double get_proba_mesure_castle(
    Coord const &king,
    Coord const &rook) const noexcept;

/**
 * @brief Vérifie si la case à une possibilité de contenir une pièce,
 * et si elle n'en a pas modifie m_piece_board en Piece().
 *
 * @param pos Les coordonnées de la position de la case à vérifier.

```

```

Coord
Coord
std::s
std::o

```



```

*/
void update_case(std::size_t pos) noexcept;
/**
 * @brief Fonction qui permet de mettre à jour le plateau,
 * car ce mouvement entraine l'apparition de plusieurs instance de board
 * identique, il faut donc les concaténer en ajoutant les probas, si la
 * proba vaut 0, on supprime l'instance
 *
 * @warning Complexité en  $k^2 \times N \times M$ , avec  $k$  la taille de  $m\_board$ 
 * c'est à dire le nombre d'instance du plateau,  $N$  et  $M$  les dimensions
 * du plateau
 */
void update_board() noexcept;
void update_board_classic() noexcept;

/**
 * @brief Mouvement classique d'une pièce qui "saute"
 * (cavalier, roi)
 *
 * @warning Ne procède aucune vérification sur la validité du mouvement
 *
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] s Coordonnées de la source du mouvement
 * @param[in] t Coordonnées de la cible du mouvement
 * @param[in] val_mes Optionel peut determiner la mesure de
 * la présence d'une pièce ou non
 */
CONSTEXPR void move_classic_jump(
    Coord const &s,

```

```

Coord const &t,
std::optional<bool> val_mes = std::nullopt);

/**
 * @brief Mouvement "split jump"
 *
 * @warning Aucun tests sur la validité du mouvement (cible vide, ect)
 *
 * @tparam N Le nombre de lignes du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] s Coordonnées de la source du mouvement
 * @param[in] t1 Coordonnées de la cible 1 (qui doit être vide)
 * @param[in] t2 Coordonnées de la cible 2 (qui doit être vide)
 */
CONSTEXPR void move_split_jump(
    Coord const &s,
    Coord const &t1,
    Coord const &t2);

/**
 * @brief Mouvement de merge
 *
 * @warning Aucun test sur la validité du mouvement
 * (cible vide, pièce identique sur les sources, ect)
 *
 * @tparam N Le nombre de lignes du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] s1 Coordonnées de la source 1
 * @param[in] s2 Coordonnées de la source 2
 * @param[in] t Coordonnées de la cible

```

```

*/
CONSTEXPR void move_merge_jump(
    Coord const &s,
    Coord const &t1,
    Coord const &t2);

/**
 * @brief Mouvement classique d'un pièce qui "glisse" (fou, dame, tour)
 *
 * @warning Ne procède aucune vérification sur la validité du mouvement
 *
 * @tparam N Le nombre de lignes
 * @tparam M Le nombre de colonnes
 * @param[in] s Coordonnées de la source
 * @param[in] t Coordonnées de la cible
 * @param[in] check_path Fonction qui permet de vérifier la présence
 * d'une pièce entre la source et la cible sur une instance du plateau
 * @param[in] val_mes Optionnel peut déterminer la mesure de
 * la présence d'une pièce ou non
 */
CONSTEXPR void
move_classic_slide(Coord const &s, Coord const &t,

```

std::f

std::o

```

/**
 * @brief Mouvement "split slide"
 * @warning Aucun tests sur la validité du mouvement (cible vide, ect)
 * @tparam N Le nombre de lignes du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] s Coordonnées de la source du mouvement
 * @param[in] t1 Coordonnées de la cible 1 (qui doit être vide)
 * @param[in] t2 Coordonnées de la cible 2 (qui doit être vide)
 * @param[in] check_path Fonction qui permet de vérifier la présence d'une
 * pièce entre la source et la cible sur une instance du plateau
 */
CONSTEXPR void move_split_slide(
    Coord const &s,
    Coord const &t1,
    Coord const &t2,
    std::function<
        bool(
            Board<N, M> const &,
            Coord const &,
            Coord const &,
            std::size_t,
            std::optional<Coord>)
        >
        check_path);

/**
 * @brief Mouvement de merge
 * @warning Aucun test sur la validité du mouvement
 * (cible vide, pièce identique sur les sources, ect)
 * @tparam N Le nombre de lignes du plateau

```

```

* @tparam M Le nombre de colonnes du plateau
* @param[in] s1 Coordonnées de la source 1
* @param[in] s2 Coordonnées de la source 2
* @param[in] t Coordonnées de la cible
* @param[in] check_path Fonction qui permet de vérifier la présence d'une
* pièce entre la source et la cible sur une instance du plateau
*/
CONSTEXPR void move_merge_slide(
    Coord const &s1,
    Coord const &s2,
    Coord const &t,
    std::function<
        bool(
            Board<N, M> const &,
            Coord const &,
            Coord const &,
            std::size_t,
            std::optional<Coord>>>
        check_path);

/**
 * @brief Le mouvement de pion d'une case fonctionne comme
 * un mouvement de jump classique, à la différence qu'un
 * pion ne peut pas capturer une pièce en avançant.
 * On mesure de la même façon qu'un jump classique en considérant
 * toutes les pièces comme des pièces alliées
 *
 * @warning Ne procède aucune vérification sur la validité du mouvement
 *
 * @tparam N Le nombre de lignes du plateau

```

```

* @tparam M Le nombre de colonnes du plateau
* @param[in] s Coordonnées de la source
* @param[in] t Coordonnées de la cible
* @param[in] val_mes Optionel peut determiner la mesure de
* la présence d'une pièce ou non
* @return true si le mouvement à etait effectué
* @return false sinon
*/

```

```

CONSTEXPR bool

```

```

move_pawn_one_step(Coord const &s, Coord const &t,

```

```

std::o

```

```

/**
 * @brief Le mouvement de pion de deux cases fonctionne
 * comme un mouvement de slide classique, à la différence
 * qu'un pion ne peut pas capturer une pièce en avançant,
 * on mesure de la même façon qu'un slide classique en considérant
 * toutes les pièces comme des pièces alliées.
 *
 * @warning Aucun test sur la possibilité de faire un mouvement de deux
 * cases du pion, pas de mise a jour du board sur la prise en passant
 *
 * @tparam N Le nombre de lignes du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] s Coordonnées de la source
 * @param[in] t Coordonnées de la cible
 * @param[in] val_mes Optionel peut determiner la mesure de
 * la présence d'une pièce ou non
 * @return true si le mouvement à etait effectué
 * @return false sinon

```



```

* @param[in] s Les coordonnées de la source
* @param[in] t Les coordonnées de la cible (l'endroit où arrive le pion)
* @param[in] ep Les coordonnées du pion capturer "en passant"
* @param[in] val_mes Optionel peut determiner la mesure de
* la présence d'une pièce ou non
* @return true si le mouvement à etait effectué
* @return false sinon
*/
CONSTEXPR bool
move_entrassant(Coord const &s, Coord const &t, Coord const &ep,
                std::optional<bool> va

/**
 * @brief Mesure la présence d'une pièce
 *
 * @tparam N Le nombre de lignes du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] position La position de la pièce mesurée
 * @param[in] val_mes Optionel peut determiner la mesure de
 * la présence d'une pièce ou non
 * @return true Si la pièce est présente sur la case position
 * @return false Sinon
 */
CONSTEXPR bool mesure(Coord const &p,

/**
 * @brief Fonction qui permet de faire une mesure dans le cas
 * d'un mouvement "capture slide"
 *

```



```

* @tparam N Le nombre de lignes du plateau
* @tparam M Le nombre de colonnes du plateau
* @param[in] s Coordonnées de la source du mouvement
* @param[in] t Coordonnées de la cible du mouvement
* @param[in] check_path Fonction qui permet de vérifier si il y a une pièce
* entre la source et la cible sur une instance du plateau
* @param[in] val_mes Optionel peut determiner la mesure de
* la présence d'une pièce ou non
* @return true Si la mesure indique de faire le mouvement
* @return false Sinon
*/
CONSTEXPR bool
mesure_capture_slide(Coord const &s, Coord const &t,

```

```

/**
* @brief Mouvement classique d'une pièce (pion exclu)
* @warning Ne procède aucune vérification sur la validité du mouvement
* @tparam N Le nombre de ligne du plateau
* @tparam M Le nombre de colonnes du plateau
* @param[in] s Coordonnées de la source du mouvement
* @param[in] t Coordonnées de la cible du mouvement
*/
CONSTEXPR void move_classic(Coord const &s, Coord const &t,

```

```

/**
 * @brief Mouvement split d'une pièce (pion exclu)
 * @warning Ne procède aucune vérification sur la validité du mouvement
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] s Coordonnées de la source du mouvement
 * @param[in] t1 Coordonnées de la cible 1 (qui doit être vide)
 * @param[in] t2 Coordonnées de la cible 2 (qui doit être vide)
 */
CONSTEXPR void move_split(Coord const &s,

```

```

/**
 * @brief Mouvement de merge de pièce (pion exclu)
 *
 * @warning Aucun test sur la validité du mouvement
 * (cible vide, pièce identique sur les sources, ect)
 *
 * @tparam N Le nombre de lignes du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] s1 Coordonnées de la source 1
 * @param[in] s2 Coordonnées de la source 2
 * @param[in] t Coordonnées de la cible
 */
CONSTEXPR void move_merge(Coord const &s1,

```

```

/**
 * @brief Gère tout les mouvements d'un pion
 * @warning Ne procède aucune vérification sur la validité du mouvement
 * @tparam N Le nombre de lignes du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] s Coordonnées de la source
 * @param[in] t Coordonnées de la cible
 * @param[in] val_mes Optionel peut determiner la mesure de
 * la présence d'une pièce ou non
 * @return true si le mouvement à etait effectué
 * @return false sinon
 */
CONSTEXPR bool move_pawn(
    Coord const &s,
    Coord const &t,
    std::optional<bool> val_mes = std::nullopt) noexcept;

/**
 * @brief Renvoie une position 1D d'une coordonnée 2D
 *
 * @param[in] ligne L'indice de ligne
 * @param[in] colonne L'indice de colonne
 * @return std::size_t La position en 1D
 */
CONSTEXPR static std::size_t
offset(std::size_t ligne,
        std::size_t colonne) noexcept;

/**
 * @brief Initialise un plateau à l'aide des listes d'initialisations

```

```

*
* @param[in] board La liste d'initialisation en 2D
* @param[out] first_instance Le tableau de la première
* instance du plateau
* @param[out] piece_board Le plateau contenant les pièces
*/
constexpr static void
initializer_list_to_2_array(
    std::initializer_list<
        std::initializer_list<
            Piece>> const &board,
    std::array<bool, N * M> &first_instance,
    std::array<Piece, N * M> &piece_board) noexcept;

/**
 * @brief Construit les mailbox en fonction
 * des dimensions du plateau
 *
 * @param[out] S_mailbox La petite mailbox de la taille du plateau
 * @param[out] L_mailbox La grande mailbox comportant 4 ligne de plus
 * et 2 colonne de plus
 * @return constexpr
 */
constexpr static void init_mailbox(std::array<int, N * M>

/**
 * @brief fonction auxiliaire qui permet de modifier un plateau

```

à l'aide d'un array de la forme du type de retour de la fonction qubitToArray, le deuxième éléments du tableau à une probabilité non nulle lors d'un mouvement split

```

*
* @tparam Q La taille du qubit
* @tparam N Le nombre de ligne du plateau
* @tparam M Le nombre de collone du plateau
* @param[in] arrayQubit Type de retour de la fonction qubitToArray,
* représente la facon dont va être modifier m_board
* @param[in] position_board L'indice du plateau dans le tableau de
* toutes les instances du plateau
* @param[in] tab_positions Tableau des indices des cases modifiées,
* on utilise N*M+1 pour signifier que le qubit en question
* est un qubit auxiliaire qui ne modifie pas le board
*/
template <std::size_t Q>
CONSTEXPR void modify(std::array<std::pair<std::array<bool, Q>,

```

```

/**
 * @brief Fonction qui permet d'effectuer un mouvement
 * sur une instance du plateau
 *
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @tparam Q La taille du qubit
 * @param[in] case_modif Permet d'initialiser le qubit

```

```

* @param[in] position L'instance du plateau modifiée
* @param[in] matrix La matrice du mouvement que l'on veut effectuer
* @param[in] tab_positions La position sur le plateau des variables
* utilisées dans le qubit, que l'on met à N*M+1 si les variables
* ne représentent pas des pièces
*/
template <std::size_t Q>
constexpr void move_1_instance(std::array<bool, Q> const &case_modif,

```

```

/**
 * @brief Mouvement du petit roque.
 * @warning Aucun test sur la validité du mouvement
 * @param[in] king Coordonnées du roi
 * @param[in] rook Coordonnées de la tour
 * @param[in] val_mes Optionel peut determiner la mesure de
 * la présence d'une pièce ou non
 */
constexpr void king_side_castle(
    Coord const &king,
    Coord const &rook,
    std::optional<bool> val_mes = std::nullopt);

/**
 * @brief Fonction qui permet de mesurer la possibilité de faire le roque
 *
 * @param[in] king Coordonnées du roi

```

```

* @param[in] rook Coordonnées de la tour
* @param[in] val_mes Optionel peut determiner la mesure de
* la présence d'une pièce ou non
* @return true si le roque est possible
* @return false sinon
*/
CONSTEXPR bool mesure_castle(
    Coord const &king,
    Coord const &rook,
    std::optional<bool> val_mes = std::nullopt);

/**
 * @brief Mouvement du grand roque.
 *
 * @warning Ne procède aucune vérification sur la validité du mouvement
 *
 * @param[in] king Coordonnées du roi
 * @param[in] rook Coordonnées de la tour
 * @param[in] val_mes Optionel peut determiner la mesure de
 * la présence d'une pièce ou non
 */
CONSTEXPR void queen_side_castle(
    Coord const &king,
    Coord const &rook,
    std::optional<bool> val_mes = std::nullopt);

/**
 * @brief Le tableau de toutes les instances possibles
 * du plateau
 */

```

```

std::vector<std::pair<std::array<bool, N * M>,

        m_board;

/**
 * @brief Le plateau indiquant le type des pièces
 */
std::array<Piece, N * M> m_piece_board;

/**
 * @brief La petite mailbox
 */
std::array<int, N * M> m_S_mailbox;

/**
 * @brief La grande mailbox
 */
std::array<int, (N + 4) * (M + 2)> m_L_mailbox;

/**
 * @brief Color::WHITE si c'est aux blanc de jouer aux noirs sinon
 */
Color m_color_current_player;

/**
 * @brief Vrai si les noirs[0] / blancs[1] peuvent faire le petit roque
 */
std::array<bool, 2> m_k_castle;

/**

```



```

    * @brief Vrai si les noirs[0] / blancs[1] peuvent faire le grand roque
    */
    std::array<bool, 2> m_q_castle;

    /**
     * @brief Contient la case sur laquelle il est
     * possible de faire une prise en passant qui
     * est vide (la case occupé si le pion avait avancé
     * d'une seule case)
     */
    std::optional<Coord> m_ep;
};

#include "Board.tpp"
#include "mesure.tpp"
#include "get_proba_move.tpp"
#include "move.tpp"

#endif

```

```

// Lib standard
#include <array>
#include <initializer_list>
#include <algorithm>
#include <cassert>
#include <utility>
#include <functional>
#include <cmath>
#include <optional>
#include <stdexcept>

// Inclusion projet
#include <Qubit.hpp>
#include <CMatrix.hpp>
#include <Unitary.hpp>
#include <TypePiece.hpp>
#include <math_utility.hpp>
#include <Constexpr.hpp>
#include <Move.hpp>
#include <Random.hpp>
#include "Board.hpp"

template <std::size_t N, std::size_t M>
constexpr Board<N, M>::Board()
    : m_board(),
      m_piece_board(),
      m_S_mailbox(),
      m_L_mailbox(),
      m_color_current_player(Color::WHITE),
      m_k_castle({true, true}),

```

```

        m_q_castle({true, true}),
        m_ep()
{
    m_board.push_back(std::pair<std::array<bool, 64>,

        init_mailbox(m_S_mailbox, m_L_mailbox);
};

template <std::size_t N, std::size_t M>
CONSTEXPR Board<N, M>::Board(std::initializer_list<

        : m_board(),
          m_piece_board(),
          m_S_mailbox(),
          m_L_mailbox(),
          m_color_current_player(Color::WHITE),
          m_k_castle({true, true}),
          m_q_castle({true, true}),
          m_ep()
{
    init_mailbox(m_S_mailbox, m_L_mailbox);
    m_board.push_back(std::pair<std::array<bool, N * M>,

        initializer_list_to_2_array(board, m_board[0].first, m_piece_board);
};

template <std::size_t N, std::size_t M>
CONSTEXPR std::size_t
Board<N, M>::offset(std::size_t ligne, std::size_t colonne) noexcept

```

```

{
    return ligne * M + colonne;
};

template <std::size_t N, std::size_t M>
CONSTEXPR std::size_t Board<N, M>::numberLines() noexcept
{
    return N;
};

template <std::size_t N, std::size_t M>
CONSTEXPR std::size_t Board<N, M>::numberColumns() noexcept
{
    return M;
};

template <std::size_t N, std::size_t M>
CONSTEXPR void
Board<N, M>::initializer_list_to_2_array(
    std::initializer_list<
        std::initializer_list<
            Piece>> const &board,
    std::array<bool, N * M> &first_instance,
    std::array<Piece, N * M> &piece_board) noexcept
{
    auto it_tab{std::begin(first_instance)};
    auto it_piece_dst{std::begin(piece_board)};
    assert(std::size(board) <= N && "Value entry out of Board");
    for (std::initializer_list<Piece> const &e : board)

```

```

{
    auto const it_tab_begin_line{it_tab};
    assert(std::size(e) <= M && "Value entry out of Board");
    std::for_each(std::begin(e),
                                                           std::end(e),
                                                           [it_tab](Piece piece) {
-> void
{

        std::move(std::begin(e), std::end(e), it_piece_dst);
        it_tab = it_tab_begin_line + M;
        it_piece_dst += M;
    }
}

template <std::size_t N, std::size_t M>
CONSTEXPR void
Board<N, M>::init_mailbox(
    std::array<int, N * M> &S_mailbox,
    std::array<int, (N + 4) * (M + 2)> &L_mailbox) noexcept
{
    std::fill(std::begin(L_mailbox),

```

```

std::begin(L_mailbox) + (2 * (M + 2) + 1), -1);

std::fill(std::end(L_mailbox) - (2 * (M + 2) + 1),
          std::end(L_mailbox), -1);

auto offset_L_mailboard{
    [](std::size_t n, std::size_t m)
        -> std::size_t
    {
        return n * (M + 2) + m;
    }
};

for (std::size_t i{0}; i < N; i++)
{
    L_mailbox[offset_L_mailboard(i + 2, 0)] = -1;
    L_mailbox[offset_L_mailboard(i + 2, M + 1)] = -1;
    for (std::size_t j{0}; j < M; j++)
    {
        L_mailbox[offset_L_mailboard(i + 2, j + 1)] =
            static_cast<int>(Board<N, M>::offset(i, j));
        S_mailbox[Board<N, M>::offset(i, j)] =
            static_cast<int>(offset_L_mailboard(i + 2, j + 1));
    }
}

}

template <std::size_t N, std::size_t M>
CONSTEXPR double Board<N, M>::get_proba(Coord const &pos) const noexcept
{
    double acc{0.};
    for (auto const &e : m_board)
    {

```

```

        acc += pow(abs(e.second), 2.) * e.first[offset(pos.n, pos.m)];
    }
    return acc;
}

template <std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
Board<N, M>::get_list_normal_move(Coord const &pos) const
{
    if ((*this)(pos.n, pos.m).get_type() != TypePiece::EMPTY)
    {
        return (*this)(pos.n, pos.m).get_list_normal_move(*this, pos);
    }
    return std::forward_list<Coord>{};
}

template <std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
Board<N, M>::get_list_split_move(Coord const &pos) const
{
    if ((*this)(pos.n, pos.m).get_type() != TypePiece::EMPTY)
    {
        return (*this)(pos.n, pos.m).get_list_split_move(*this, pos);
    }
    return std::forward_list<Coord>{};
}

template <std::size_t N, std::size_t M>
CONSTEXPR Color Board<N, M>::get_current_player() const noexcept
{

```

```

        return m_color_current_player;
    }

    template <std::size_t N, std::size_t M>
    CONSTEXPR void Board<N, M>::change_player() noexcept
    {
        if (get_current_player() == Color::WHITE)
        {
            m_color_current_player = Color::BLACK;
        }
        else
        {
            m_color_current_player = Color::WHITE;
        }
    }
}

template <std::size_t N, std::size_t M>
template <std::size_t Q>
CONSTEXPR void Board<N, M>::modify(
    std::array<
        std::pair<
            std::array<bool, Q>,
            std::complex<double>>,
            2> const &arrayQubit,
    std::size_t position_board,
    std::array<std::size_t, Q> const &tab_positions)
{
    if (!complex_equal(arrayQubit[1].second, 0i))
    {

```



```

std::pair<std::array<bool, N * M>, std::complex<double>> new_b{};
std::copy(std::begin(m_board[position_board].first),
          std::end(m_board[position_board].first),
          std::begin(new_b.first));
new_b.second = m_board[position_board].second;
for (std::size_t i{0}; i < Q; i++)
{
    if (tab_positions[i] < N * M + 1)
    { /*Ca nous permet de mettre des variables
      dans les qubits qui ne sont pas prises
      en compte lors de la modif du plateau */
      new_b.first[tab_positions[i]] = arrayQubit[1].first[i];
    }
    new_b.second *= arrayQubit[1].second;
    m_board.push_back(new_b);
}
for (std::size_t i{0}; i < Q; i++)
{
    if (tab_positions[i] < N * M + 1)
    {
        m_board[position_board].first[tab_positions[i]] =
            arrayQubit[0].first[i];
    }
}
m_board[position_board].second *= arrayQubit[0].second;
}

template <std::size_t N, std::size_t M>
template <std::size_t Q>

```

```

CONSTEXPR void
Board<N, M>::move_1_instance(std::array<bool, Q> const &case_modif,

{
    Qubit<Q> q{case_modif};
    auto x{qubitToArray(matrix * q)};
    modify(std::move(x), position, tab_positions);
}

template <std::size_t N, std::size_t M>
CONSTEXPR Piece const &
Board<N, M>::operator()(std::size_t n, std::size_t m) const noexcept
{
    return m_piece_board[offset(n, m)];
}

template <std::size_t N, std::size_t M>
void Board<N, M>::update_board() noexcept
{
    std::size_t size_board = std::size(m_board);
    for (std::size_t i{size_board}; i > 0; i--)
    {
        using namespace std::complex_literals;
        if (complex_equal(m_board[i - 1].second, 0i))
        {
            m_board.erase(std::begin(m_board) + i - 1);
        }
    }
}

```

```

        else
        {
            for (std::size_t j{i - 1}; j > 0; j--)
            {
                if (m_board[i - 1].first == m_board[j - 1].first)
                {
                    m_board[j - 1].second += m_board[i - 1].second;
                    m_board.erase(std::begin(m_board) + i - 1);
                    break;
                }
            }
        }
    }

template <std::size_t N, std::size_t M>
constexpr bool Board<N, M>::winning_position(Color c) const noexcept
{
    for (std::size_t i{0}; i < N * M; i++)
    {
        if (m_piece_board[i].get_type() == TypePiece::KING &&
            m_piece_board[i].get_color() != c)
        {
            return false;
        }
    }
    return true;
}

template <std::size_t N, std::size_t M>
constexpr bool

```

```

Board<N, M>::move_is_legal(Move const &move) const
{
    if (move.type == TypeMove::NORMAL)
    {
        auto list{get_list_normal_move(move.normal.src)};
        return std::find(
                                std::begin(list),
                                std::end(list),
                                move.normal.arv) !=
                                std::end(list);
    }
    else if (move.type == TypeMove::SPLIT)
    {
        std::forward_list<Coord> list{get_list_split_move(move.split.src)};
        std::array<bool, 2> found{};
        std::array<Coord, 2> arv{{move.split.arv1, move.split.arv2}};
        for (Coord const &e : list)
        {
            for (std::size_t i{0}; i < std::size(arv); i++)
            {
                if (e == arv[i])
                {
                    found[i] = true;
                }
            }
        }
        if (std::all_of(
                                std::begin(found),
                                std::end(found),
                                [](bool v) -> bool
                                { return v; })))
    }
}

```

```

        {
            break;
        }
    }
    else if (move.type == TypeMove::MERGE)
    {
        std::forward_list<Coord> list1{
            get_list_split_move(move.merge.src1)};
        std::forward_list<Coord> list2{
            get_list_split_move(move.merge.src2)};

        return std::find(
            std::begin(list1),
            std::end(list1),
            move.merge.arv) !=
            std::end(list1) &&
            std::find(
            std::begin(list2),
            std::end(list2),
            move.merge.arv) !=
            std::end(list2);
    }
}

template <std::size_t N, std::size_t M>
void Board<N, M>::update_case(std::size_t pos) noexcept
{
    std::size_t size_board{std::size(m_board)};
    for (std::size_t i{0}; i < size_board; i++)

```

```

    {
        if (m_board[i].first[pos])
        {
            return;
        }
    }
    m_piece_board[pos] = Piece();
}

template <std::size_t N, std::size_t M>
constexpr bool
Board<N, M>::check_if_use_move_promote(Coord const &pos) const noexcept
{
    return (*this)(pos.n, pos.m).check_if_use_move_promote(*this, pos);
}

template <std::size_t N, std::size_t M>
constexpr std::forward_list<Move>
Board<N, M>::get_list_promote(Coord const &pos) const noexcept
{
    return (*this)(pos.n, pos.m).get_list_promote(*this, pos);
}

template <std::size_t N, std::size_t M>
template <class UnitaryFunction>
constexpr void
Board<N, M>::all_move(
    UnitaryFunction func,
    std::optional<Color> color_player) const noexcept
{

```

```
std::unordered_map<TypePiece,
```

```
std::unordered
```

```
move_merge{};
```

```
for (std::size_t i{0}; i < numberLines(); i++)
```

```
{
```

```
    for (std::size_t j{0}; j < numberColumns(); j++)
```

```
    {
```

```
        Piece const &piece{(*this)(i, j)};
```

```
        double p_proba{get_proba(Coord(i, j))};
```

```
        if (piece.get_type() != TypePiece::EMPTY &&  
            piece.get_color() == color_player
```

```
        {
```

```
            if (!check_if_use_move_promote(Coord(i, j)))
```

```
            {
```

```
                std::forward_list<Coord> move_normal{
```

```
                    get_list_normal_move(Coord(i, j))};
```

```
                for (Coord const &c : move_normal)
```

```
                {
```

```
                    if (func(Move_classic(Coord(i, j), c)))
```

```
                    {
```

```
                        return;
```

```
                    }
```

```
                }
```

```

std::forward_list<Coord> move_split{
    get_list_split_move(Coord(i, j))};

std::forward_list<Coord>::
    const_iterator it_move{
        std::cbegin(move_split)}
for (; it_move != std::cend(move_split);
    it_move++)
{
    for (std::forward_list<Coord>::
        const_iterator
            it2 != cend(move_split); it2++
        {
            if (it_move == it2)
            {
                continue;
            }
            if (func(Move_split(Coord(i, j), *it_mo
            {
                return;
            }
        }
    if (piece.get_type() != TypePiece::PAWN)
    {
        if (move_merge
            .contai
        {
            if (move_merge

```



```

{
    for (Coord const &c :
        move_m

    {
        if (p_proba < 1

        {
            if (fun

            {
            }

        }

    }
}
else
{
    move_merge

    .at(pie
    .insert

```

```

    }
    }
    else
    {
        move_merge
        .insert(

    }
}

}
}
}
else
{
    std::forward_list<Move> list{
        get_list_promote(Coord(i, j))};

    for (Move const &move : list)
    {
        if (func(move))
        {

```

```

    }
}

return;
}
}
}
}

template <std::size_t N, std::size_t M>
void Board<N, M>::update_board_classic() noexcept
{
    std::size_t size_board = std::size(m_board);
    for (std::size_t i{size_board}; i > 0; i--)
    {
        using namespace std::complex_literals;
        if (complex_equal(m_board[i - 1].second, 0i))
        {
            m_board.erase(std::begin(m_board) + i - 1);
        }
        else
        {
            for (std::size_t j{i - 1}; j > 0; j--)
            {
                if (m_board[i - 1].first == m_b
                    {
                        double inter =
                            m_board[j - 1].
                            m_board.erase(s
                                break;

```

}

}

}

}

}

}

}

```
template <std::size_t N, std::size_t M>
void Board<N, M>::update_board_classic() noexcept
{
    std::size_t size_board = std::size(m_board);
    for (std::size_t i{size_board}; i > 0; i--)
    {
        using namespace std::complex_literals;
        if (complex_equal(m_board[i - 1].second, 0i))
        {
            m_board.erase(std::begin(m_board) + i - 1);
        }
        else
        {
            for (std::size_t j{i - 1}; j > 0; j--)
            {
                if (m_board[i - 1].first == m_b
                    {
                        double inter =
                            m_board[j - 1].
                            m_board.erase(s
                                break;
```

```
void Board<N, M>::update_board_classic() noexcept
```

f

```
std::size_t size_board = std::size(m_board);
```

```
for (std::size_t i{size_board}; i > 0; i--)
```

{

```
using namespace std::complex_literals;
```

```
if (complex_equal(m_board[i - 1].second, 0i))
```

{

```
m_board.erase(std::begin(m_board) + i - 1);
```

}

```
else
```

f

```
for (std::size_t j{i - 1}; j > 0; j--)
```

{

```
if (m_board[i - 1].first == m_b
```

{

```
double inter =
```

```
m_board[j - 1].
```

```
m_board.erase(s
```

```
break;
```

```

    }
}

}

}

/*std::size_t n_size_board = std::size(m_board);
double all_proba {0};
for (std::size_t i{0}; i<n_size_board; i++)
{
    all_proba += std::pow(std::abs(m_board[i].second), 2);
}
all_proba = std::pow(all_proba, 1./2);
for (std::size_t i{0}; i<n_size_board; i++)
{
    m_board[i].second /= all_proba;
}*/
}

```

```

// Lib standard
#include <cassert>
#include <utility>
#include <functional>
#include <cmath>
#include <optional>
#include <stdexcept>

// Inclusion projet
#include <Piece.hpp>
#include <TypePiece.hpp>
#include <Constexpr.hpp>
#include "Board.hpp"

template <std::size_t N, std::size_t M>
CONSTEXPR bool Board<N, M>::mesure(Coord const &p,

{

    Piece p_actuelle = (*this)(p.n, p.m);
    if (p_actuelle.get_type() == TypePiece::EMPTY)
    {
        return false;
    }
    else
    {
        bool mes;
        if (val_mes == std::nullopt)
        {
            double x = rnd::randreal(0., 1.);

```

```

std::size_t indice_mes = 0;
double pow_coef{
    std::pow(
        std::abs(m_board[indice_mes]),
        std::size_t size_board{std::size(m_board)});
while (x - pow_coef > 0 && indice_mes < size_board)
{
    x -= pow_coef;
    indice_mes++;
    if (indice_mes >= size_board)
    {
        throw std::runtime_error("indice_mes >= size_board");
    }
    pow_coef = std::pow(std::abs(m_board[indice_mes]),
        std::size_t size_board);
}
mes = m_board[indice_mes].first[offset(p.n, p.m)];
}
else
{
    mes = val_mes.value();
}
double proba_delete = 0;
// std::size_t nbr_elt_suppr{0};
for (std::size_t i{std::size(m_board)}; i > 0; i--)
{
    if (m_board[i - 1].first[offset(p.n, p.m)] != m_board[i].first[offset(p.n, p.m)])
    {
        proba_delete +=
            std::pow(std::abs(m_board[i - 1].first[offset(p.n, p.m)]),
                std::size_t size_board);
    }
}

```

```

        m_board.erase(
                                std::begin(m_bo

        }

    }
    for (auto &e : m_board)
    {
        e.second /= std::sqrt(1. - proba_delete);
    }
    for (std::size_t i{0}; i < N * M; i++)
    {
        if (p_actuelle.get_type() != TypePiece::EMPTY)
        {
            update_case(i);
        }
    }
    return mes;
}

}

template <std::size_t N, std::size_t M>
CONSTEXPR bool
Board<N, M>::mesure_capture_slide(
    Coord const &s,
    Coord const &t,
    std::function<bool(Board<N, M> const &,

Coord
Coord
std::s
std::o

check_path,

```

```

{
    std::optional<bool> val_mes)

    std::size_t position = offset(s.n, s.m);
    Piece p_actuelle = (*this)(s.n, s.m);
    if (p_actuelle.get_type() == TypePiece::EMPTY)
    {
        return false;
    }
    else
    {
        bool mes;
        if (val_mes == std::nullopt)
        {
            double x = rnd::randreal(0., 1.);
            std::size_t indice_mes = 0;
            double pow_coef{
                std::pow(std::abs(m_board[0].se
            std::size_t size_board{std::size(m_board)};
            while (x - pow_coef > 0 && indice_mes < size_bo
            {
                x -= pow_coef;
                indice_mes++;
                if (indice_mes >= size_board)
                {
                    throw std::run
                }
                pow_coef = std::pow(std::abs(m_

                // indice_suppr++,
            }
        }
    }
}

```



```

        mes = m_board[indice_mes].first[position] &&
            check_path(*this, s, t,
    }
    else
    {
        mes = val_mes.value();
    }
    double proba_delete = 0;
    // std::size_t nbr_elt_suppr{0};
    for (std::size_t i{std::size(m_board)}; i > 0; i--)
    {
        if ((m_board[i - 1].first[position] &&
            check_path(*this, s, t, i - 1,
        {
            proba_delete +=
                std::pow(std::a

            m_board.erase(
                std::begin(m_bo

        }
    }
    for (auto &e : m_board)
    {
        e.second /= std::sqrt(1. - proba_delete);
    }
    for (std::size_t i{0}; i < N * M; i++)
    {
        if (p_actuelle.get_type() != TypePiece::EMPTY)
        {
            update_case(i);

```

```

    }
    }
    return mes;
}

}

template <std::size_t N, std::size_t M>
constexpr bool
Board<N, M>::mesure_castle(
    Coord const &king,
    Coord const &rook,
    std::optional<bool> val_mes)
{
    std::size_t position_king = offset(king.n, king.m);
    std::size_t position_rook = offset(rook.n, rook.m);

    bool mes;
    if (val_mes == std::nullopt)
    {
        double x = rnd::randreal(0., 1.);
        std::size_t indice_mes = 0;
        std::size_t size_board{std::size(m_board)};
        double pow_coef{
            std::pow(std::abs(m_board[0].second), 2)};

        while (x - pow_coef > 0 && indice_mes < size_board - 1)
        {
            x -= pow_coef;
            indice_mes++;
            if (indice_mes >= size_board)

```

```

        {
            throw std::runtime_error("Indic
        }
        pow_coef = std::pow(std::abs(m_board[indice_mes
    }
    mes = m_board[indice_mes].first[position_king] &&
        m_board[indice_mes].first[position_rook] &&
        check_path_straight_1_instance(*this, k
}
else
{
    mes = val_mes.value();
}
double proba_delete = 0;
// std::size_t nbr_elt_suppr{0};
for (std::size_t i{std::size(m_board)}; i > 0; i--)
{
    if ((m_board[i - 1].first[position_king] &&
        m_board[i - 1].first[position_rook] &&
        check_path_straight_1_instance(*this, king, ro
    {
        proba_delete +=
            std::pow(std::abs(m_board[i - 1
        m_board.erase(
            std::begin(m_board) + i - 1);
    }
}
for (auto &e : m_board)
{

```

```

        e.second /= std::sqrt(1. - proba_delete);
    }
    for (std::size_t i{0}; i < N * M; i++)
    {
        if (m_piece_board[i].get_type() != TypePiece::EMPTY)
        {
            update_case(i);
        }
    }
    return mes;
}

```

```

// Lib standard
#include <array>
#include <algorithm>
#include <cassert>
#include <utility>
#include <functional>
#include <cmath>
#include <optional>
#include <stdexcept>

// Inclusion projet
#include <Piece.hpp>
#include <CMatrix.hpp>
#include <Unitary.hpp>
#include <TypePiece.hpp>
#include <math_utility.hpp>
#include <Constexpr.hpp>
#include <Move.hpp>
#include "Board.hpp"

template <std::size_t N, std::size_t M>
CONSTEXPR void Board<N, M>::king_side_castle(Coord const &k,

{
    std::size_t king = offset(k.n, k.m);
    std::size_t new_king = offset(k.n, k.m + 2);
    std::size_t rook = offset(r.n, r.m);
    std::size_t new_rook = offset(r.n, r.m - 2);
    if (mesure_castle(k, r, val_mes))

```

```

{
    for (std::size_t i{0}; i < std::size(m_board); i++)
    {
        move_1_instance(
            std::array<bool, 2>{true, false},
            MATRIX_ISWAP,
            std::array<std::size_t, 2>{king, rook});
        move_1_instance(
            std::array<bool, 2>{true, false},
            MATRIX_ISWAP,
            std::array<std::size_t, 2>{rook, king});
        m_piece_board[new_king] = std::move(m_piece_board[king]);
        m_piece_board[new_rook] = std::move(m_piece_board[rook]);
        m_piece_board[king] = Piece();
        m_piece_board[rook] = Piece();
    }
}

template <std::size_t N, std::size_t M>
constexpr void Board<N, M>::queen_side_castle(Coord const &k,
                                                Coord const &r,
                                                int val_mes)
{
    std::size_t king = offset(k.n, k.m);
    std::size_t new_king = offset(k.n, k.m - 2);
    std::size_t rook = offset(r.n, r.m);
    std::size_t new_rook = offset(r.n, r.m + 3);
    if (mesure_castle(k, r, val_mes))
    {
        for (std::size_t i{0}; i < std::size(m_board); i++)

```

```

        {
            move_1_instance(
                std::array<bool, 2>{true, false},
                MATRIX_ISWAP,
                std::array<std::size_t, 2>{king, rook});
            move_1_instance(
                std::array<bool, 2>{true, false},
                MATRIX_ISWAP,
                std::array<std::size_t, 2>{rook, king});
        }
        m_piece_board[new_king] = std::move(m_piece_board[king]);
        m_piece_board[new_rook] = std::move(m_piece_board[rook]);
        m_piece_board[king] = Piece();
        m_piece_board[rook] = Piece();
    }

template <std::size_t N, std::size_t M>
constexpr void
Board<N, M>::move_classic_jump(Coord const &s, Coord const &t,
{
    std::size_t source = offset(s.n, s.m);
    std::size_t target = offset(t.n, t.m);

    if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
        (m_piece_board[target].get_type() == m_piece_board[source].get_type() &
         m_piece_board[target].get_color() == m_piece_board[source].get_color()))
    {
        std::size_t const size_board{std::size(m_board)};
        for (std::size_t i{0}; i < size_board; i++)

```

```

{
    move_1_instance(
        std::array<bool, 2>{m_board[i].
        MATRIX_ISWAP,
        std::array<std::size_t, 2>{sour
    }
    m_piece_board[target] = std::move(m_piece_board[source]);
    m_piece_board[source] = Piece();
}
else
{
    if (m_piece_board[source].same_color(m_piece_board[target]))
    {
        std::optional<bool> negation ;
        if(val_mes.has_value())
        {
            negation = !val_mes.value();
        }
        if (!mesure(t, negation))
        {
            for (std::size_t i{0}; i < std::
            {
                move_1_instance

            }
            m_piece_board[target] = std::mo
            m_piece_board[source] = Piece()
        }
    }
}

```



```

    }
    else
    {
        if (mesure(s, val_mes))
        {
            for (std::size_t i{0}; i < std::mo
            {
                CONTEXPR CMatr

            move_1_instance

        }
        m_piece_board[target] = std::mo
        m_piece_board[source] = Piece()
    }
}
}
}
}

```

```

template <std::size_t N, std::size_t M>
CONSTEXPR bool
Board<N, M>::move_pawn_one_step(Coord const &s, Coord const &t,

{

    std::size_t source = offset(s.n, s.m);
    std::size_t target = offset(t.n, t.m);

    if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
        (m_piece_board[target].get_type() == m_piece_board[source].get_type()
         & m_piece_board[target].get_color() == m_piece_board[source].get_color()))
    {
        std::size_t const size_board{std::size(m_board)};
        for (std::size_t i{0}; i < size_board; i++)
        {
            move_1_instance(
                std::array<bool, 2>{m_board[i].n, m_board[i].m},
                i, MATRIX_ISWAP,
                std::array<std::size_t, 2>{source, target});
        }
        m_piece_board[target] = std::move(m_piece_board[source]);
        m_piece_board[source] = Piece();
        return true;
    }
    else
    {
        std::optional<bool> negation ;
        if (val_mes.has_value())
        {

```

```

        negation = !val_mes.value();
    }
    if (!measure(t, negation))
    {
        for (std::size_t i{0}; i < std::size(m_board);
            {
                move_1_instance(
                    std::array<bool, N>{
                        i, MATRIX_ISWAP,
                    },
                    std::array<std::size_t, N>{
                        t.n, t.m,
                    })
                m_piece_board[target] = std::move(m_piece_board[source]);
                m_piece_board[source] = Piece();
                return true;
            }
        }
        return false;
    }
}

template <std::size_t N, std::size_t M>
CONSTEXPR bool
Board<N, M>::move_pawn_two_step(Coord const &s, Coord const &t,
    {
        std::size_t source = offset(s.n, s.m);
        std::size_t target = offset(t.n, t.m);
        if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
            (m_piece_board[target].get_type() == m_piece_board[source].get_type()
             && m_piece_board[target].get_color() == m_piece_board[source].get_color()))
            return false;
        m_piece_board[target] = std::move(m_piece_board[source]);
        m_piece_board[source] = Piece();
        return true;
    }
}

```

```

{
    std::size_t const size_board{std::size(m_board)};
    for (std::size_t i{0}; i < size_board; i++)
    {
        move_1_instance(
            std::array<bool, 3>{!check_path(
                i, MATRIX_SLIDE,
                std::array<std::size_t, 3>{N *
            }
            m_piece_board[target] = m_piece_board[source];
            update_case(source);
            return true;
        }
    else
    {
        std::optional<bool> negation ;
        if (val_mes.has_value())
        {
            negation = !val_mes.value();
        }
        if (!mesure(t, negation))
        {
            for (std::size_t i{0}; i < std::size(m_board);
            {
                move_1_instance(
                    std::array<bool, 3>{!check_path(

```

```

        i, MATRIX_SLIDE
        std::array<std::
    }
    m_piece_board[target] = std::move(m_piece_board
    update_case(source);
    return true;
}
}
return false;
}

template <std::size_t N, std::size_t M>
CONSTEXPR bool Board<N, M>::capture_pawn(
    Coord const &s,
    Coord const &t,
    std::optional<bool> val_mes)
{
    std::size_t source = offset(s.n, s.m);
    std::size_t target = offset(t.n, t.m);
    if (measure(s, val_mes))
    {
        for (std::size_t i{0}; i < std::size(m_board); i++)
        {
            if (m_board[i].first[target])
            {
                CONSTEXPR CMatrix<8> A{
                    MATRIX_JUMP

```

```

CMatrix<2>::ide

move_1_instance(
    std::array<bool, 2> a,

    i, A,
    std::array<std::size_t, 2> t,

    }
}
m_piece_board[target] = m_piece_board[source];
update_case(source);
return true;
}
return false;
}

template <std::size_t N, std::size_t M>
CONSTEXPR bool
Board<N, M>::move_enpassant(Coord const &s, Coord const &t, Coord const &ep,
{
    std::size_t source = offset(s.n, s.m);
    std::size_t target = offset(t.n, t.m);
    std::size_t enpassant = offset(ep.n, ep.m);

    if (m_piece_board[target].get_type() == TypePiece::EMPTY ||

```

```

        (m_piece_board[target].get_type() == m_piece_board[source].get_type() &&
         m_piece_board[target].get_color() == m_piece_board[source].get_color())
    {

        std::size_t const size_board{std::size(m_board)};
        for (std::size_t i{0}; i < size_board; i++)
        {
            if (m_board[i].first[enpassant])
            {
                // permet d'enlever le pion pris
                move_1_instance(
                    std::array<bool, 2>,
                    i, MATRIX_ISWAP);
                move_1_instance(
                    std::array<bool, 2>,
                    i, MATRIX_ISWAP);
            }

            m_piece_board[target] = std::move(m_piece_board[source]);
            m_piece_board[source] = Piece();
            m_piece_board[enpassant] = Piece();
            return true;
        }
    }
    else
    {
        if (m_piece_board[source].same_color(m_piece_board[target]))
        {
            std::optional<bool> negation ;

```

```

if(val_mes.has_value())
{
    negation = !val_mes.value();
}
if (!mesure(t, negation))
{

    for (std::size_t i{0}; i < std:
    {
        if (m_board[i].
        {

    }

}
m_piece_board[target] = m_piece
m_piece_board[source] = Piece()
return true;

```



```
    }  
    return false;  
}  
else  
{  
  
    if (mesure(s, val_mes))  
    {  
        for (std::size_t i{0}; i < std:  
        {  
            if (m_board[i].  
            {  
                if (m_board[i].
```

```

    }

    m_piece_board[target] = std::mo
    m_piece_board[source] = Piece()
    m_piece_board[enpassant] = Piec
    return true;
}

}

}

return false;
}

template <std::size_t N, std::size_t M>
CONSTEXPR void
Board<N, M>::move_split_jump(Coord const &s, Coord const &t1, Coord const &t2)
{
    std::size_t const source = offset(s.n, s.m);
    std::size_t const target1 = offset(t1.n, t1.m);
    std::size_t const target2 = offset(t2.n, t2.m);
    std::size_t const size_board{std::size(m_board)};
    for (std::size_t i{0}; i < size_board; i++)
    {
        move_1_instance(
            std::array<bool, 3>{
                m_board[i].first[target2],
                m_board[i].first[target1],

```

```

                                m_board[i].first[source]],
                                i, MATRIX_SPLIT,
                                std::array<std::size_t, 3>{target2, target1, so
}
m_piece_board[target1] = m_piece_board[source];

if (m_piece_board[target1].get_type() == TypePiece::EMPTY &&
    m_piece_board[target2].get_type() == TypePiece::EMPTY)
{
    m_piece_board[target2] = std::move(m_piece_board[source]);
    m_piece_board[source] = Piece();
    update_case(target1);
    update_case(target2);
}
else
{
    m_piece_board[target2] = m_piece_board[source];
    update_case(source);
    update_case(target1);
    update_case(target2);
    update_board();
}
}

template <std::size_t N, std::size_t M>
CONSTEXPR void
Board<N, M>::move_merge_jump(Coord const &s1, Coord const &s2, Coord const &t)
{
    std::size_t source1 = offset(s1.n, s1.m);
    std::size_t source2 = offset(s2.n, s2.m);

```

```

std::size_t target = offset(t.n, t.m);
std::size_t const size_board{std::size(m_board)};
for (std::size_t i{0}; i < size_board; i++)
{
    move_1_instance(
        std::array<bool, 3>{
            m_board[i].first[source1],
            m_board[i].first[source2],
            m_board[i].first[target]},
        i, MATRIX_MERGE,
        std::array<std::size_t, 3>{source1, source2, ta
    }
    m_piece_board[target] = m_piece_board[source1];
    update_board();
    update_case(target);
    update_case(source1);
    update_case(source2);
}

template <std::size_t N, std::size_t M>
CONSTEXPR void
Board<N, M>::move_classic_slide(
    Coord const &s,
    Coord const &t,
    std::function<
        bool(
            Board<N, M> const &,
            Coord const &,
            Coord const &,
            std::size_t,

```

```

                                std::optional<Coord>>>
                                check_path,
                                std::optional<bool> val_mes)
{
    std::size_t source = offset(s.n, s.m);
    std::size_t target = offset(t.n, t.m);
    if (m_piece_board[target].get_type() == TypePiece::EMPTY ||
        (m_piece_board[target].get_type() == m_piece_board[source].get_type()
         && m_piece_board[target].get_color() == m_piece_board[source].get_color()))
    {
        std::size_t const size_board{std::size(m_board)};
        for (std::size_t i{0}; i < size_board; i++)
        {
            move_1_instance(
                std::array<bool, 3>{
                    !check_path(
                        false,
                        m_board[i].first,
                        i, MATRIX_SLIDE,
                        std::array<std::size_t, 3>{N *
                    }
                },
                m_piece_board[target] = m_piece_board[source];
                update_case(source);
            }
        }
        else
        {
            if (m_piece_board[source].same_color(m_piece_board[target]))
            {
                std::optional<bool> negation ;
            }
        }
    }
}

```

```

        if(val_mes.has_value())
        {
            negation = !val_mes.value();
        }
        if (!mesure(t, negation))
        {
            for (std::size_t i{0}; i < std::size(m_piece_board); ++i)
            {
                move_1_instance(m_piece_board[i], t, negation);
            }
            m_piece_board[target] = std::move(m_piece_board[source]);
            m_piece_board[source] = Piece();
        }
    }
    else
    {
        if (mesure_capture_slide(s, t, check_path, val_mes))
        {
            for (std::size_t i{0}; i < std::size(m_piece_board); ++i)
            {
                auto const A{

```

```

move_1_instance

}

m_piece_board[target] = std::mo
m_piece_board[source] = Piece()

}

}

}

}

template <std::size_t N, std::size_t M>
CONSTEXPR void
Board<N, M>::move_split_slide(
    Coord const &s,
    Coord const &t1,
    Coord const &t2,
    std::function<
        bool(
            Board<N, M> const &,
            Coord const &,
            Coord const &,
            std::size_t,
            std::optional<Coord>>>
        check_path)

```

```
{
```

```
std::size_t source = offset(s.n, s.m);
std::size_t target1 = offset(t1.n, t1.m);
std::size_t target2 = offset(t2.n, t2.m);
std::size_t const size_board{std::size(m_board)};
for (std::size_t i{0}; i < size_board; i++)
{
    move_1_instance(
        std::array<bool, 5>{
            !check_path(*this, s, t2, i, t1),
            !check_path(*this, s, t1, i, t2),
            m_board[i].first[target2],
            m_board[i].first[target1],
            m_board[i].first[source]},
        i, MATRIX_SPLIT_SLIDE,
        std::array<std::size_t, 5>{
            N * M + 1,
            N * M + 1,
            target2,
            target1,
            source});
    /*La matrice split slide est créée pour
    être utilisé sans appliquer de porte cnot a
    un chemin, nos fonctions check_path renvoie
    un résultat où l'on a appliquer la porte cnot */
}
if (m_piece_board[target1].get_type() == TypePiece::EMPTY &&
    m_piece_board[target2].get_type() == TypePiece::EMPTY)
{
    m_piece_board[target1] = m_piece_board[source];
}
```



```

        m_piece_board[target2] = m_piece_board[source];
        update_case(source);
        update_case(target1);
        update_case(target2);
    }
    else
    {
        m_piece_board[target1] = m_piece_board[source];
        m_piece_board[target2] = m_piece_board[source];
        update_case(source);
        update_case(target1);
        update_case(target2);
        update_board();
    }
}

template <std::size_t N, std::size_t M>
CONSTEXPR void
Board<N, M>::move_merge_slide(
    Coord const &s1,
    Coord const &s2,
    Coord const &t,
    std::function<
        bool(
            Board<N, M> const &,
            Coord const &,
            Coord const &,
            std::size_t,
            std::optional<Coord>>>
        check_path)

```

```
{
```

```
    std::size_t const source1 = offset(s1.n, s1.m);
    std::size_t const source2 = offset(s2.n, s2.m);
    std::size_t const target = offset(t.n, t.m);
    std::size_t const size_board{std::size(m_board)};
    for (std::size_t i{0}; i < size_board; i++)
    {
        move_1_instance(
            std::array<bool, 5>{
                !check_path(*this, s2, t, i, s1),
                !check_path(*this, s1, t, i, s2),
                m_board[i].first[source1],
                m_board[i].first[source2],
                m_board[i].first[target]],
            i, MATRIX_MERGE_SLIDE,
            std::array<std::size_t, 5>{
                N * M + 1,
                N * M + 1,
                source1,
                source2,
                target});
        /* La matrice merge slide est créée pour
           être utilisé sans appliquer de porte cnot au chemin,
           nos fonctions check_path renvoie un résultat où l'on
           a appliqué la porte cnot */
    }
    m_piece_board[target] = m_piece_board[source1];
    update_board();
    update_case(target);
    update_case(source1);
}
```

```

        update_case(source2);
    }

    template <std::size_t N, std::size_t M>
    CONSTEXPR bool Board<N, M>::move_pawn(
        Coord const &s,
        Coord const &t,
        std::optional<bool> val_mes) noexcept
    {
        bool move_exec{false};
        auto abs_diff{[] (std::size_t x, std::size_t y) -> std::size_t
                        {
                            return
                                diff_line{abs_diff(s.n, t.n)};
                                diff_col{abs_diff(s.m, t.m)};
                        }};

        std::size_t diff_line{abs_diff(s.n, t.n)};
        if (diff_line == 2)
        {
            move_exec = move_pawn_two_step(s, t, val_mes);
            m_ep = Coord((s.n + t.n) / 2, s.m);
        }
        else if (diff_line == 1)
        {
            std::size_t diff_col{abs_diff(s.m, t.m)};
            if (diff_col == 1)
            {
                if (m_ep != std::nullopt && m_ep == t)
                {
                    int step{
                        ((*this)(s.n, s.m) == t) ? 1 : -1;
                    }
                }
            }
        }
    }

```

Color::WHITE)

```
Coord ep;
ep.m = m_ep->m;
ep.n = m_ep->n - step;
move_exec = move_enpassant(s, t
    }
    else
    {
        move_exec = capture_pawn(s, t,
    }
}
else
{
    move_exec = move_pawn_one_step(s, t, val_mes);
}
m_ep = std::nullopt;
}
return move_exec;
}

template <std::size_t N, std::size_t M>
constexpr void
Board<N, M>::move_promotion(
    Move const &move,
    std::optional<bool> val_mes)
{
    TypePiece p{move.promote.piece};
    Coord s{move.promote.src};
```

```

Coord t{move.promote.arv};

if (p == TypePiece::QUEEN ||
    p == TypePiece::KNIGHT ||
    p == TypePiece::BISHOP ||
    p == TypePiece::ROOK)
{
    if (move_pawn(s, t, val_mes))
    {
        Piece piece{p, m_piece_board[offset(t.n, t.m)]}.
        m_piece_board[offset(t.n, t.m)] = std::move(piece);
    }
}
else
{
    throw std::runtime_error("Piece non valide pour faire une promo");
}
}

template <std::size_t N, std::size_t M>
CONSTEXPR void
Board<N, M>::move_classic(
    Coord const &s,
    Coord const &t,
    std::optional<bool> val_mes)
{
    TypePiece piece{(*this)(s.n, s.m).get_type()};
    switch (piece)
    {
        case TypePiece::KING:

```

```

if constexpr (N == 8 && M == 8)
{
    auto abs_diff{[] (std::size_t x, std::size_t y)

        std::size_t diff_colum{abs_diff(s.m, t.m)};
        if (diff_colum == 2)
        {
            if (s.m > t.m)
            {
                queen_side_castl(s, t, val_mes);
            }
            else
            {
                king_side_castl(s, t, val_mes);
            }
        }
        else
        {
            move_classic_jump(s, t, val_mes);
        }
    }
    else
    {
        move_classic_jump(s, t, val_mes);
    }
    break;
case TypePiece::KNIGHT:
    move_classic_jump(s, t, val_mes);
}

```

```

        break;
    case TypePiece::BISHOP:
        move_classic_slide(
            s, t,
            &check_path_diagonal_1_instance<N, M>,
            val_mes);
        break;
    case TypePiece::ROOK:
        move_classic_slide(
            s, t,
            &check_path_straight_1_instance<N, M>,
            val_mes);
        break;
    case TypePiece::QUEEN:
        move_classic_slide(
            s, t,
            &check_path_queen_1_instance<N, M>,
            val_mes);
        break;
    case TypePiece::PAWN:
        move_pawn(s, t, val_mes);
        return;
    case TypePiece::EMPTY:
    default:
        break;
}
m_ep = std::nullopt;
}

template <std::size_t N, std::size_t M>

```

```

CONSTEXPR void Board<N, M>::move_split(Coord const &s,

{
    TypePiece piece{(*this)(s.n, s.m).get_type()};
    switch (piece)
    {
        case TypePiece::KING:
        case TypePiece::KNIGHT:
            move_split_jump(s, t1, t2);
            break;
        case TypePiece::BISHOP:
            move_split_slide(s, t1, t2, &check_path_diagonal_1_instance<N,
            break;
        case TypePiece::ROOK:
            move_split_slide(s, t1, t2, &check_path_straight_1_instance<N,
            break;
        case TypePiece::QUEEN:
            move_split_slide(s, t1, t2, &check_path_queen_1_instance<N, M>)
            break;
        case TypePiece::PAWN:
        case TypePiece::EMPTY:
        default:
            break;
    }
    m_ep = std::nullopt;
}

template <std::size_t N, std::size_t M>
CONSTEXPR void Board<N, M>::move_merge(Coord const &s1,

```



```
{
```

```
    TypePiece piece1{(*this)(s1.n, s1.m).get_type()};  
    TypePiece piece2{(*this)(s2.n, s2.m).get_type()};  
    if (piece1 != piece2)  
    {  
        return;  
    }  
    switch (piece1)  
    {  
    case TypePiece::KING:  
    case TypePiece::KNIGHT:  
        move_merge_jump(s1, s2, t);  
        break;  
    case TypePiece::BISHOP:  
        move_merge_slide(s1, s2, t, &check_path_diagonal_1_instance<N,  
        break;  
    case TypePiece::ROOK:  
        move_merge_slide(s1, s2, t, &check_path_straight_1_instance<N,  
        break;  
    case TypePiece::QUEEN:  
        move_merge_slide(s1, s2, t, &check_path_queen_1_instance<N, M>)  
        break;  
    case TypePiece::PAWN:  
    case TypePiece::EMPTY:  
    default:  
        break;  
    }  
    m_ep = std::nullopt;
```

```

}

template <std::size_t N, std::size_t M>
CONSTEXPR void Board<N, M>::move(Move const &movement, std::optional<bool> val_mes)
{
    if (std::empty(m_board))
    {
        throw std::runtime_error("Le plateau est vide impossible de réa
    }
    switch (movement.type)
    {
    case TypeMove::NORMAL:
        move_classic(movement.normal.src, movement.normal.arv, val_mes);
        break;
    case TypeMove::SPLIT:
        move_split(movement.split.src,
                    movement.split.arv1,
                    movement.split.arv2);
        break;
    case TypeMove::MERGE:
        move_merge(movement.merge.src1,
                    movement.merge.src2,
                    movement.merge.arv);
        break;
    case TypeMove::PROMOTE:
        move_promotion(movement, val_mes);
        break;
    default:
        break;
    }
}

```

```
}  
    update_board_classic();  
    //update_board();
```

```

// Lib standard
#include <cassert>
#include <utility>
#include <functional>
#include <cmath>
#include <optional>
#include <stdexcept>

// Inclusion projet
#include <Piece.hpp>
#include <TypePiece.hpp>
#include <Constexpr.hpp>
#include "Board.hpp"
#include <math_utility.hpp>

template <std::size_t N, std::size_t M>
CONSTEXPR double
Board<N, M>::get_proba_mesure(Coord const &p) const noexcept
{
    double res{0};
    std::size_t size_board = std::size(m_board);
    for (std::size_t i{0}; i < size_board; i++)
    {
        if (m_board[i].first[offset(p.n, p.m)])
        {
            res += std::pow(std::abs(m_board[i].second), 2)
        }
    }
    return res;
}

```

```

template <std::size_t N, std::size_t M>
CONSTEXPR double
Board<N, M>::get_proba_mesure_capture_slide(
    Coord const &s,
    Coord const &t,
    std::function<bool(Board<N, M> const &,
                                Coord
                                Coord
                                std::s
                                std::o

    check_path) const noexcept
{
    double res{0};
    std::size_t position = offset(s.n, s.m);
    std::size_t size_board = std::size(m_board);
    for (std::size_t i{0}; i < size_board; i++)
    {
        if (m_board[i].first[position] &&
            check_path(*this, s, t, i, std::nullopt))
        {
            res += std::pow(std::abs(m_board[i].second), 2)
        }
    }
    return res;
}

template <std::size_t N, std::size_t M>
CONSTEXPR double
Board<N, M>::get_proba_mesure_castle(

```

```

        Coord const &king,
        Coord const &rook) const noexcept
{
    double res{0};
    std::size_t size_board = std::size(m_board);
    std::size_t position_king = offset(king.n, king.m);
    std::size_t position_rook = offset(rook.n, rook.m);
    for (std::size_t i{0}; i < size_board; i++)
    {
        if (m_board[i].first[position_king] &&
            m_board[i].first[position_rook] &&
            check_path_straight_1_instance(*this, king, rook, i))
        {
            res += std::pow(std::abs(m_board[i].second), 2);
        }
    }
    return res;
}

template <std::size_t N, std::size_t M>
constexpr double Board<N, M>::get_proba_move(Move const &move) const noexcept
{
    Coord s;
    Coord t;
    switch (move.type)
    {
    case TypeMove::NORMAL:
        s = move.normal.src;
        t = move.normal.arv;
        break;
    }
}

```

```

case TypeMove::PROMOTE:
    s = move.promote.src;
    t = move.promote.arv;
    break;
case TypeMove::MERGE:
    return 1.;
case TypeMove::SPLIT:
    return 1.;
default:
    return 1.;
}
TypePiece piece{(*this)(s.n, s.m).get_type()};
TypePiece piece_target{(*this)(t.n, t.m).get_type()};
if((piece == piece_target) && (*this)(s.n, s.m).same_color((*this)(t.n, t.m)))
{
    return 1.;
}
switch (piece)
{
case TypePiece::KING:
    if constexpr (N == 8 && M == 8)
    {
        auto abs_diff{[] (std::size_t x, std::size_t y)

        std::size_t diff_colum{abs_diff(s.m, t.m)};
        if (diff_colum == 2)
        {
            if (s.m > t.m)

```

```

        {
            return get_prob
        }
        else
        {
            return get_prob
        }
    }
    else
    {
        if (piece_target != TypePiece::
        {
            if ((*this)(s.n
            {
            }
            else
            {
            }

        }
        else
        {
            return 1.;
        }
    }
}
else
{
    if (piece_target != TypePiece::EMPTY)

```



```

        {
            if ((*this)(s.n, s.m).same_color(t.n, t.m))
            {
                return 1. - get_proba_mesure(t);
            }
            else
            {
                return get_proba_mesure(s);
            }
        }
        else
        {
            return 1.;
        }
    }
    break;
case TypePiece::KNIGHT:
    if (piece_target != TypePiece::EMPTY)
    {
        if ((*this)(s.n, s.m).same_color((*this)(t.n, t.m))
        {
            return 1. - get_proba_mesure(t);
        }
        else
        {
            return get_proba_mesure(s);
        }
    }
    else
    {

```

```

        return 1.;
    }

    break;
case TypePiece::BISHOP:
    if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY)
    {
        return 1.;
    }
    else
    {
        if (!(*this)(t.n, t.m).same_color((*this)(s.n,
        {
            return get_proba_mesure_capture
                s, t,
                &check_path_dia

        })
        else
        {
            return 1. - get_proba_mesure(t)

        }
    }
    break;
case TypePiece::ROOK:
    if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY)
    {
        return 1.;
    }
    else
    {

```

```

        if (!(*this)(t.n, t.m).same_color((*this)(s.n,
        {
            return get_proba_mesure_capture
                s, t,
                &check_path_str
        }
        else
        {
            return 1. - get_proba_mesure(t)
        }
    }
    break;
case TypePiece::QUEEN:
    if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY)
    {
        return 1.;
    }
    else
    {
        if (!(*this)(t.n, t.m).same_color((*this)(s.n,
        {
            return get_proba_mesure_capture
                s, t,
                &check_path_que
        }
        else
        {
            return 1. - get_proba_mesure(t)
        }
    }
}

```

```

        break;
    case TypePiece::PAWN:
        if ((*this)(t.n, t.m).get_type() == TypePiece::EMPTY ||
            ((*this)(t.n, t.m).get_type() == (*this)(s.n, s.m).get_type() &&
            (*this)(t.n, t.m).get_color() == (*this)(s.n, s.m).get_color())
        {
            return 1.;
        }
        else
        {
            return 1. - get_proba_mesure(t);
        }
    case TypePiece::EMPTY:
    default:
        break;
    }
    return 1.;
}

```

```

#ifndef PIECE_HPP
#define PIECE_HPP

#include <Color.hpp>
#include <Coord.hpp>
#include <concepts>
#include <vector>
#include <forward_list>
#include <constexpr.hpp>
#include <Move.hpp>
#include "TypePiece.hpp"

template <std::size_t N, std::size_t M>
class Board;

/**
 * @brief Class conservant le type et la couleur d'une pièce.
 * Elle permet aussi de recuperer ses mouvement possible
 */
class Piece
{
public:
    constexpr inline Piece() noexcept;

    /**
     * @brief Construit une piece à l'aide de son type et de sa couleur
     *
     * @param[in] piece Le type de la piece
     * @param[in] color La couleur de la piece
     */

```

```

constexpr inline Piece(TypePiece piece, Color color) noexcept;
constexpr inline Piece(Piece const &) = default;
constexpr inline Piece &operator=(Piece const &) = default;

/**
 * @brief Déplacement d'un objet piece vers un autre objet piece
 */
constexpr inline Piece(Piece &&) = default;
constexpr inline Piece &operator=(Piece &&) = default;
constexpr inline ~Piece() = default;

/**
 * @brief Renvoie le type de la piece
 *
 * @return Une énumération TypePiece
 */
constexpr inline TypePiece get_type() const noexcept;

/**
 * @brief Renvoie la couleur d'un piece
 *
 * @return Color La couleur de la piece
 */
constexpr inline Color get_color() const noexcept;

/**
 * @brief Récupère la liste des mouvements classiques pour une piece
 *
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonne du plateau

```

```

* @param[in] board Le plateau
* @param[in] pos la position de la piece sur le plateau
* @warning Le resultat n'est pas garanti si la position ne concorde pas
* avec la position de la piece sur le plateau
* @return std::forward_list<Coord> La liste de coordonnées des positions
* d'arrivées de chaque mouvement possible
*/
template <std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
get_list_normal_move(Board<N, M> const &board,

/**
 * @brief Récupère la liste des mouvements split pour une piece
 *
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonne du plateau
 * @param[in] board Le plateau
 * @param[in] pos la position de la piece sur le plateau
 * @warning Le resultat n'est pas garanti si la position ne concorde pas
 * avec la position de la piece sur le plateau
 * @warning Ne renvoie qu'une liste de coordonnées uniques car
 * l'ensemble des mouvements possibles est toutes les couples
 * de coordonnées de la liste
 * @return std::forward_list<Coord> La liste de coordonnées
 * des positions d'arrivées possible sachant que chaque élément
 * représente une seule des deux cases d'arrivées d'un split move
 */
template <std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>

```

```

get_list_split_move(Board<N, M> const &board,

/**
 * @brief Teste si la piece est blanche
 *
 * @return bool true si la piece est blanche, false sinon
 */
constexpr inline bool is_white() const noexcept;

/**
 * @brief Teste si la piece est noire
 *
 * @return bool true si la piece est noire, false sinon
 */
constexpr inline bool is_black() const noexcept;

/**
 * @brief Teste si l'autre piece est de la meme couleur
 *
 * @param[in] other L'autre pièce à comparer
 *
 * @return bool true si la piece est de la meme couleur, false sinon
 */
constexpr inline bool same_color(Piece const &other) const noexcept;

/**
 * @brief Teste si les mouvement de la piece sont si la
 * pièce est un pion un mouvement de promotion
 *

```



```
private:
```

```

/**
 * @brief Renvoie la liste de toutes les promotions
 *
 * @warning Ne verifie pas la validité du mouvement,
 * de la position ou du type de la pièce
 *
 * @param[in] board Le plateau
 * @param[in] pos La position du pion
 * @return La liste de tout les mouvements de promotion
 */

```

Coord **const** &p

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

```

* @brief Énumération utilisé dans le template des fonctions
* qui récupère les listes de mouvement pour modifier leurs
* comportement lors de la prise de pièce.
*/
enum class Move_Mode
{
    /**
     * @brief Dans le cas du mouvement classique on
     * autorise la prise de pièce.
     */
    NORMAL,

    /**
     * @brief Dans le cas du split on interdit la
     * prise de pièce.
     */
    SPLIT
};

/**
 * @brief renvoie la valeur absolue d'une soustraction sur des types size_t
 *
 * @param[in] x Variable de type size_t
 * @param[in] y Variable de type size_t
 * @return std::size_t Retourne l'opération |x - y|
 */
constexpr inline static std::size_t
abs_substracte(std::size_t x, std::size_t y) noexcept;

/**

```

```

* @brief renvoie la distance entre deux coordonnées
* à l'aide de la norme 2
*
* @param[in] x Première coordonnée
* @param[in] y Seconde coordonnée
* @return double le résultat de la norme 1
*/
constexpr inline static double
norm(Coord const &x, Coord const &y) noexcept;

/**
 * @brief Récupère la liste de mouvement d'une pièce pour
 * les déplacements indiqué de manière récursive
 * c'est à dire tout les mouvements infini (ex : diagonale du fou)
 *
 * @tparam MOVE Le type de mouvement utiliser entre normal
 * et split (split ne prend pas les mouvements de capture de pièce)
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonne du plateau
 * @tparam SIZE La taille de la liste des mouvements possible
 * @param[in] board Le plateau
 * @param[in] pos La position de la pièce
 * @param[in] list_move La liste de tout les mouvements possible
 * @return std::forward_list<Coord> La liste de coordonnées
 * des positions d'arrivées possible
 */
template <Move_Mode MOVE, std::size_t N, std::size_t M, std::size_t SIZE>
CONSTEXPR std::forward_list<Coord>
get_list_move_rec(Board<N, M> const &board, Coord const &pos,

```

std::ar

```

/**
 * @brief Récupère la liste de mouvement du roi
 *
 * @warning Ne récupère les mouvements du roque que pour un tableau
 * de taille 8 x 8
 *
 * @warning ne verifie pas si c'est bien un roi
 *
 * @tparam MOVE Le type de mouvement utiliser entre normal
 * et split (split ne prend pas les mouvements de capture de pièce)
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonne du plateau
 * @param[in] board Le plateau
 * @param[in] pos La position du roi
 * @return std::forward_list<Coord> La liste de coordonnées
 * des positions d'arrivées possible pour le roi
 */
template <Move_Mode MOVE, std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
get_list_move_king(Board<N, M> const &board,

```

Coord

```

/**
 * @brief Récupère la liste de mouvement du cavalier
 *
 * @warning ne verifie pas si c'est bien un cavalier
 *
 * @tparam MOVE Le type de mouvement utiliser entre normal
 * et split (split ne prend pas les mouvements de capture de pièce)

```

```

* @tparam N Le nombre de ligne du plateau
* @tparam M Le nombre de colonne du plateau
* @param[in] board Le plateau
* @param[in] pos La position du cavalier
* @return std::forward_list<Coord> La liste de coordonnées
* des positions d'arrivées possible pour le cavalier
*/
template <Move_Mode MOVE, std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
get_list_move_knight(Board<N, M> const &board,

/**
* @brief Récupère la liste de mouvement du fou
*
* @warning ne verifie pas si c'est bien un fou
*
* @tparam MOVE Le type de mouvement utiliser entre normal
* et split (split ne prend pas les mouvements de capture de pièce)
* @tparam N Le nombre de ligne du plateau
* @tparam M Le nombre de colonne du plateau
* @param[in] board Le plateau
* @param[in] pos La position du fou
* @return std::forward_list<Coord> La liste de coordonnées
* des positions d'arrivées possible pour le fou
*/
template <Move_Mode MOVE, std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
get_list_move_bishop(Board<N, M> const &board,

```



```

* @param[in] pos La position de la dame
* @return std::forward_list<Coord> La liste de coordonnées
* des positions d'arrivées possible pour la dame
*/
template <Move_Mode MOVE, std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
get_list_move_queen(Board<N, M> const &board,

/**
 * @brief Récupère la liste de mouvement du pion
 *
 * @warning ne verifie pas si c'est bien un pion
 *
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonne du plateau
 * @param[in] board Le plateau
 * @param[in] pos La position du pion
 * @return std::forward_list<Coord> La liste de coordonnées
 * des positions d'arrivées possible pour le pion
 */
template <std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
get_list_move_pawn(Board<N, M> const &board,

/**
 * @brief Récupère la liste de mouvement pour
 * n'importe qu'elle pièce
 *

```

Coord

```

    * @tparam MOVE Le type de mouvement utiliser entre normal
    * et split (split ne prend pas les mouvements de capture de pièce)
    * @tparam N Le nombre de ligne du plateau
    * @tparam M Le nombre de colonne du plateau
    * @param[in] board Le plateau
    * @param[in] pos La position de la pièce
    * @param[in] list_move La liste de tout les mouvements possible
    * @return std::forward_list<Coord> La liste de coordonnées
    * des positions d'arrivées possible
    */
template <Move_Mode MOVE, std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
get_list_move(Board<N, M> const &board,
                                   Coord const &pos) const

/**
 * @brief La couleur de la pièce (WHITE/BLACK)
 */
Color m_color;

/**
 * @brief Le type de la piece
 * ( KING / QUEEN / ROOK / BISHOP / KNIGHT / PAWN )
 */
TypePiece m_type;
};

#include "Piece.hpp"
#include "get_list_move.hpp"

```



```

/**
 * @brief L'objet représentant le roi blanc
 */
constexpr Piece W_KING{TypePiece::KING, Color::WHITE};

/**
 * @brief L'objet représentant le roi noir
 */
constexpr Piece B_KING{TypePiece::KING, Color::BLACK};

/**
 * @brief L'objet représentant la renne blanche
 */
constexpr Piece W_QUEEN{TypePiece::QUEEN, Color::WHITE};

/**
 * @brief L'objet représentant la renne noire
 */
constexpr Piece B_QUEEN{TypePiece::QUEEN, Color::BLACK};

/**
 * @brief L'objet représentant le fou blanc
 */
constexpr Piece W_BISHOP{TypePiece::BISHOP, Color::WHITE};

/**
 * @brief L'objet représentant le fou noir
 */
constexpr Piece B_BISHOP{TypePiece::BISHOP, Color::BLACK};

```

```

/**
 * @brief L'objet représentant la tour blanche
 */
constexpr Piece W_ROOK{TypePiece::ROOK, Color::WHITE};

/**
 * @brief L'objet représentant la tour noire
 */
constexpr Piece B_ROOK{TypePiece::ROOK, Color::BLACK};

/**
 * @brief L'objet représentant le cavalier blanc
 */
constexpr Piece W_KNIGHT{TypePiece::KNIGHT, Color::WHITE};

/**
 * @brief L'objet représentant le cavalier noir
 */
constexpr Piece B_KNIGHT{TypePiece::KNIGHT, Color::BLACK};

/**
 * @brief L'objet représentant le pion blanc
 */
constexpr Piece W_PAWN{TypePiece::PAWN, Color::WHITE};

/**
 * @brief L'objet représentant le pion noir
 */
constexpr Piece B_PAWN{TypePiece::PAWN, Color::BLACK};

```

#endif

```

#include "Piece.hpp"
#include <Coord.hpp>
#include <check_path.hpp>
#include <cmath>
#include <forward_list>
#include <observer_ptr.hpp>
#include <math_utility.hpp>
#include <Constexpr.hpp>
#include <Move.hpp>

#define POW2(x) ((x) * (x))

constexpr inline Piece::Piece() noexcept
    : m_color(Color::BLACK),
      m_type(TypePiece::EMPTY)
{
}

constexpr inline Piece::Piece(TypePiece piece, Color color) noexcept
    : m_color(color),
      m_type(piece)
{
}

constexpr inline bool Piece::is_white() const noexcept
{
    return m_color == Color::WHITE;
}

constexpr inline bool Piece::is_black() const noexcept

```

```

{
    return m_color == Color::BLACK;
}

constexpr inline bool
Piece::same_color(Piece const &other) const noexcept
{
    return m_color == other.m_color;
}

constexpr inline TypePiece Piece::get_type() const noexcept
{
    return m_type;
}

constexpr inline Color Piece::get_color() const noexcept
{
    return m_color;
}

constexpr inline std::size_t
Piece::abs_substracte(std::size_t x, std::size_t y) noexcept
{
    if (x >= y)
    {
        return x - y;
    }
    return y - x;
}

```

```

constexpr inline double
Piece::norm(Coord const &x, Coord const &y) noexcept
{
    return sqrt(
        static_cast<double>(POW2(abs_substracte(x.n, y.n)) +

}

template <std::size_t N, std::size_t M>
CONSTEXPR bool
Piece::check_if_use_move_promote(Board<N, M> const &board,
{
    if constexpr (N >= 2)
    {
        if (board(pos.n, pos.m).get_type() == TypePiece::PAWN)
        {
            auto sign_color{
                [](Color color) -> int
                {
                    return (color =
                });
            std::size_t required_line{(get_color() == Color
            if (pos.n == required_line)
            {
                return true;
            }
        }
    }
    return false;
}

```

}

```

#include "Piece.hpp"
#include <Coord.hpp>
#include <check_path.hpp>
#include <cmath>
#include <forward_list>
#include <observer_ptr.hpp>
#include <math_utility.hpp>
#include <Constexpr.hpp>
#include <Move.hpp>

template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
Piece::get_list_move_king(Board<N, M> const &board,

{
    CONSTEXPR int P{M + 2};
    std::array<int, 8> const
        move_king{{-P - 1, -P, -P + 1, -1, 1, P - 1, P, P + 1}};
    std::size_t current_pos{board.offset(pos.n, pos.m)};
    int posBox{board.m_S_mailbox[current_pos]};
    std::forward_list<Coord> list_move{};
    for (int const e : move_king)
    {
        int arv{board.m_L_mailbox[posBox + e]};
        if (arv >= 0)
        {
            std::size_t n{arv / M}, m{arv % M};
            Piece const &arvPiece{board(n, m)};
            bool eval {false};
            if CONSTEXPR (MOVE == Move_Mode::NORMAL)

```



```

        {
            eval = arvPiece.get_type() == T
                !same_
                board.
        }
        #if !defined(KING_NO_SPLIT)
        else
        {
            eval = arvPiece.get_type() == T
        }
        #endif
        if (eval)
        {
            list_move.push_front(Coord(n, m)
        }
    }
}
if CONSEXPR (N == 8 && M == 8 && MOVE == Move_Mode::NORMAL)
{
    if (board.m_q_castle[static_cast<int>(get_color())] &&
        check_path_straight(board, pos, Coord(pos.n, pos.m - 2)))
    {
        list_move.push_front(Coord(pos.n, pos.m - 2));
    }
    if (board.m_k_castle[static_cast<int>(get_color())] &&
        check_path_straight(board, pos, Coord(pos.n, pos.m + 2)))
    {
        list_move.push_front(Coord(pos.n, pos.m + 2));
    }
}
}

```

```

        return list_move;
    }

    template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
    CONSTEXPR std::forward_list<Coord>
    Piece::get_list_move_knight(Board<N, M> const &board,

    {
        CONSTEXPR int P{M + 2};
        std::array<int, 8> const
            move_knight{{-2 * P - 1, -P - 2, -2 * P + 1, -P + 2,
                                                                    P - 2, 2 * P -

        std::size_t current_pos{board.offset(pos.n, pos.m)};
        int posBox{board.m_S_mailbox[current_pos]};
        std::forward_list<Coord> list_move{};
        for (int const e : move_knight)
        {
            int arv{board.m_L_mailbox[posBox + e]};
            if (arv >= 0)
            {
                std::size_t n{arv / M}, m{arv % M};
                Piece const &arvPiece{board(n, m)};
                bool eval;
                if CONSTEXPR (MOVE == Move_Mode::NORMAL)
                {
                    eval = arvPiece.get_type() == T
                        !same_
                        board.

                }
                else

```

```

        {
            eval = arvPiece.get_type() == T
        }
        if (eval)
        {
            list_move.push_front(Coord(n, m))
        }
    }
}

return list_move;
}

template <Piece::Move_Mode MOVE,
          std::size_t N,
          std::size_t M,
          std::size_t SIZE>
constexpr std::forward_list<Coord>
Piece::get_list_move_rec(
    Board<N, M> const &board,
    Coord const &pos,
    std::array<int, SIZE> const &list_move) const noexcept
{
    std::size_t current_pos{board.offset(pos.n, pos.m)};
    int const posBox{board.m_S_mailbox[current_pos]};
    std::forward_list<Coord> move{};
    for (int const e : list_move)
    {
        int arv{board.m_L_mailbox[posBox + e]};
        int posBox_tmp{posBox};
        while (arv >= 0)
    }
}

```

```
{
```

```
std::size_t n{arv / M}, m{arv % M};  
Piece const &arvPiece{board(n, m)};  
if CONSTEXPR (MOVE == Move_Mode::NORMAL)  
{  
    if (arvPiece.get_type() == Type  
        board.get_proba  
    {  
        move.push_front  
    }  
    else if (!same_color(arvPiece))  
    {  
        move.push_front  
        break;  
    }  
    else  
    {  
        break;  
    }  
}  
else  
{  
    if (arvPiece.get_type() == Type  
        (arvPiece.get_t  
        arvPiece.get_c  
    {  
        move.push_front  
    }  
    else  
    {
```

```

                                                    break;
                                                }
                                            }
                                        }
                                }
                                return move;
        }

template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
Piece::get_list_move_bishop(Board<N, M> const &board,
{
    CONSTEXPR int P{M + 2};
    std::array<int, 4> const move_bishop{{-P - 1, -P + 1, P - 1, P + 1}};
    return get_list_move_rec<MOVE>(board, pos, move_bishop);
}

template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
Piece::get_list_move_rook(Board<N, M> const &board,
{
    CONSTEXPR int P{M + 2};
    std::array<int, 4> const move_rook{{-P, 1, P, -1}};
    return get_list_move_rec<MOVE>(board, pos, move_rook);
}

```

```

template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
Piece::get_list_move_queen(Board<N, M> const &board,

{
    CONSTEXPR int P{M + 2};
    std::array<int, 8> const
        move_queen{{-P - 1, -P, -P + 1, -1, 1, P - 1, P, P + 1}};
    return get_list_move_rec<MOVE>(board, pos, move_queen);
}

template <std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
Piece::get_list_move_pawn(Board<N, M> const &board,

{
    CONSTEXPR int P{M + 2};
    auto sign_color{
        [] (Color color)
        {
            return (color == Color::WHITE) ? -1 : 1;
        }};
    int sign{sign_color(get_color())};
    std::array<int, 3> move_pawn{sign * P, sign * P - 1, sign * P + 1};
    std::forward_list<Coord> list_move{};
    std::size_t current_pos{board.offset(pos.n, pos.m)};
    int posBox{board.m_S_mailbox[current_pos]};
    int arv{board.m_L_mailbox[posBox + move_pawn[0]]};
    if (arv >= 0)
    {

```

```

std::size_t n{arv / N}, m{arv % N};
if (board(n, m).get_type() == TypePiece::EMPTY ||
    board.get_proba(Coord(n, m)) < 1. - EPSILON)
{
    list_move.push_front(Coord(n, m));
    if ((pos.n == 1 && get_color() == Color::BLACK
        (pos.n == N - 2 && get_color()
        (board(n + sign, m).get_type()
        board.get_proba(Coord(n + sign

    {
        list_move.push_front(Coord(n +

    }

}

for (std::size_t i{1}; i < 3; i++)
{
    arv = board.m_L_mailbox[posBox + move_pawn[i]];
    if (arv >= 0)
    {
        std::size_t n{arv / N}, m{arv % N};
        if ((board(n, m).get_type() != TypePiece::EMPTY
            !same_color(board(n, m))) ||
            (board.m_ep != std::nullopt &&
            board.m_ep->n == pos.n &&
            board.m_ep->m == pos.m))

        {
            list_move.push_front(Coord(n, m

        }

    }

}

```

```

        return list_move;
    }

    template <std::size_t N, std::size_t M>
    CONSTEXPR std::forward_list<Move>
    Piece::get_list_promote(Board<N, M> const &board, Coord const &pos) const noexcept
    {
        std::forward_list<Coord> list_move{get_list_move_pawn(board, pos)};
        std::forward_list<Move> list_promote;
        std::array<TypePiece, 4> promote_choice{TypePiece::QUEEN,

        for (Coord const &e : list_move)
        {
            for (TypePiece p : promote_choice)
            {
                list_promote.push_front(Move_promote(pos, e, p))
            }
        }
        return list_promote;
    }

    template <Piece::Move_Mode MOVE, std::size_t N, std::size_t M>
    CONSTEXPR std::forward_list<Coord>
    Piece::get_list_move(Board<N, M> const &board, Coord const &pos) const noexcept
    {
        switch (get_type())
        {
        case TypePiece::PAWN:

```



```

        if CONSTEXPR (MOVE == Move_Mode::NORMAL)
        {
            return get_list_move_pawn(board, pos);
        }
        else
        {
            return std::forward_list<Coord>{};
        }
    case TypePiece::KNIGHT:
        return get_list_move_knight<MOVE>(board, pos);
    case TypePiece::BISHOP:
        return get_list_move_bishop<MOVE>(board, pos);
    case TypePiece::ROOK:
        return get_list_move_rook<MOVE>(board, pos);
    case TypePiece::QUEEN:
        return get_list_move_queen<MOVE>(board, pos);
    case TypePiece::KING:
        return get_list_move_king<MOVE>(board, pos);
    case TypePiece::EMPTY:
    default:
        return std::forward_list<Coord>{};
    }
}

template <std::size_t N, std::size_t M>
CONSTEXPR std::forward_list<Coord>
Piece::get_list_normal_move(Board<N, M> const &board,

{
    return get_list_move<Move_Mode::NORMAL>(board, pos);
}

```

```
}  
  
template <std::size_t N, std::size_t M>  
CONSTEXPR std::forward_list<Coord>  
Piece::get_list_split_move(Board<N, M> const &board,  
  
{  
    return get_list_move<Move_Mode::SPLIT>(board, pos);  
}
```

```
#ifndef TYPE_PIECE_HPP
#define TYPE_PIECE_HPP

/**
 * @brief Enumère toutes les sortes de pièces
 */
enum class TypePiece
{
    EMPTY,
    PAWN,
    KNIGHT,
    BISHOP,
    ROOK,
    QUEEN,
    KING
};

#endif
```

```

#ifndef COORD_HPP
#define COORD_HPP

#include <cstddef>

/**
 * @brief La structure qui représente une coordonnée
 * sur le plateau
 */
struct Coord
{
    Coord() = default;
    Coord(std::size_t i, std::size_t j) : n(i), m(j) {};
    Coord(Coord const&) = default;
    Coord& operator=(Coord const&) = default;
    Coord(Coord &&) = default;
    Coord& operator=(Coord &&) = default;

    /**
     * @brief l'indice sur les lignes
     */
    std::size_t n;

    /**
     * @brief l'indice sur les colonnes
     */
    std::size_t m;
};

```

```
struct Coord_hash
{
    std::size_t operator () (Coord const &coord) const;
};

bool operator==(Coord const &lhs, Coord const &rhs);

#endif
```

```

#include "Coord.hpp"
#include <functional>

std::size_t Coord_hash::operator () (Coord const &coord) const
{
    std::size_t h1 = std::hash<std::size_t>()(coord.n);
    std::size_t h2 = std::hash<std::size_t>()(coord.m);

    return h1 ^ h2;
}

bool operator==(Coord const &lhs, Coord const &rhs)
{
    return lhs.n == rhs.n && lhs.m == rhs.m;
}

```

```

#ifdef MOVE_HPP
#define MOVE_HPP

#include <Coord.hpp>
#include <TypePiece.hpp>

/**
 * @brief Énumération représentant tout les types de mouvements
 */
enum class TypeMove
{
    /**
     * @brief Représente le mouvement classic
     */
    NORMAL,

    /**
     * @brief Représente le mouvement de split de deux pièces
     */
    SPLIT,

    /**
     * @brief Représente le mouvement de fusion de deux pièces
     */
    MERGE,

    /**
     * @brief Représente le mouvement de promotion du pion
     */
}

```

```

        PROMOTE
    };

    /**
     * @brief Stoque tout les mouvements possibles
     */
    struct Move
    {
        Move() = default;
        Move(Move const &) = default;
        Move &operator=(Move const &) = default;
        Move(Move &&) = default;
        Move &operator=(Move &&) = default;
        /**
         * @brief Indique quel est le mouvement stocké par l'union
         */
        TypeMove type;

        /**
         * @brief Stoque au choix l'un des trois mouvements possible
         */
        union
        {
            /**
             * @brief Stoque les coordonnées pour un mouvement classic
             */
            struct
            {
                /**
                 * @brief La coordonnée de depart

```



```

    */
    Coord src;

    /**
     * @brief La coordonnées d'arrivée
     */
    Coord arv;
} normal;

/**
 * @brief stocke l'ensemble des coordonnées pour un
 * mouvement splité
 */
struct
{
    /**
     * @brief La coordonnée de départ
     */
    Coord src;

    /**
     * @brief Une coordonnée d'arrivée
     */
    Coord arv1;

    /**
     * @brief Une seconde coordonnée d'arrivée
     *
     */
    Coord arv2;
}

```

```

} split;

/**
 * @brief stocke l'ensemble des coordonnées pour un
 * mouvement de fusion
 */
struct
{
    /**
     * @brief La coordonnée de la première pièce à fusionner
     */
    Coord src1;

    /**
     * @brief La coordonnée de la seconde pièce à fusionner
     */
    Coord src2;

    /**
     * @brief La coordonnée de la position d'arrivée
     */
    Coord arv;
} merge;

/**
 * @brief stocke l'ensemble des coordonnées et la pièce choisi
 * pour un mouvement de promotion
 */
struct
{

```

```

        /**
         * @brief La case de depart
         */
        Coord src;

        /**
         * @brief La case d'arrivée
         */
        Coord arv;

        /**
         * @brief La piece choisi pour le move
         */
        TypePiece piece;
    } promote;
};

bool operator==(Move const &m) const noexcept;

constexpr inline Move Move_classic(Coord const &src, Coord const &arv)
{
    Move move;
    move.type = TypeMove::NORMAL;
    move.normal.src = src;
    move.normal.arv = arv;
    return move;
}

constexpr inline Move Move_split(Coord const &src, Coord const &arv1, Coord const &arv2)

```

```

{
    Move move;
    move.type = TypeMove::SPLIT;
    move.split.src = src;
    move.split.arv1 = arv1;
    move.split.arv2 = arv2;
    return move;
}

constexpr inline Move Move_merge(Coord const &src1, Coord const &src2, Coord const &arv)
{
    Move move;
    move.type = TypeMove::MERGE;
    move.merge.src1 = src1;
    move.merge.src2 = src2;
    move.merge.arv = arv;
    return move;
}

constexpr inline Move Move_promote(Coord const &src, Coord const &arv, TypePiece promotion)
{
    Move move;
    move.type = TypeMove::PROMOTE;
    move.promote.src = src;
    move.promote.arv = arv;
    move.promote.piece = promotion;
    return move;
}

#endif

```

```

#include <Coord.hpp>
#include <TypePiece.hpp>
#include "Move.hpp"

bool Move::operator==(Move const &m) const noexcept
{
    if (type == m.type)
    {
        switch (type)
        {
            case TypeMove::NORMAL:
                return normal.src == m.normal.src &&
                    normal.arv == m.normal.arv;

            case TypeMove::SPLIT:
                return split.src == m.split.src &&
                    split.arv1 == m.split.arv1 &&
                    split.arv2 == m.split.arv2;

            case TypeMove::MERGE:
                return merge.src1 == m.merge.src1 &&
                    merge.src2 == m.merge.src2 &&
                    merge.arv == m.merge.arv;

            case TypeMove::PROMOTE:
                return promote.src == m.promote.src &&
                    promote.arv == m.promote.arv &&
                    promote.piece == m.promote.piece;

            default:
                return false;
        }
    }
    return false;
}

```

}

```

#ifdef QUBIT_HPP
#define QUBIT_HPP

#include <cstddef>
#include <complex>
#include <array>
#include <CMatrix.hpp>
#include <Constexpr.hpp>

/**
 * @brief Représente un qubit
 *
 * @tparam N La taille du Qubit
 */
template <std::size_t N>
class Qubit final
{
public:
    // Constructeur
    CONSTEXPR Qubit() = default;

    /**
     * @brief Construit un nouveau qubit à l'aide d'un tableau de booléen
     *
     * @param data le tableau de n booléen utilisé pour
     * construire un n-qubit ex : |10>
     */
    CONSTEXPR Qubit(std::array<bool, N> const &data);
    CONSTEXPR Qubit(std::array<std::complex<double>, _2POW(N)> &&init_list);

```

```

// Copie
CONSTEXPR Qubit(Qubit const &) = delete;
CONSTEXPR Qubit &operator=(Qubit const &) = delete;

// Mouvement
CONSTEXPR Qubit(Qubit &&) = delete;
CONSTEXPR Qubit &operator=(Qubit &&) = delete;

// Destructeur
CONSTEXPR ~Qubit() = default;

template <std::size_t M>
friend CONSTEXPR Qubit<M> operator*(
    CMatrix<_2POW(M)> const &lhs,
    Qubit<M> const &rhs);

template <std::size_t M>
friend std::ostream &operator<<(
    std::ostream &out,
    Qubit<M> const &qubit);

template <std::size_t M>
friend CONSTEXPR std::array<
    std::pair<
        std::array<bool, M>,
        std::complex<double>>,
        2>
    qubitToArray(Qubit<M> const &qubit);

private:

```



```

        std::array<std::complex<double>, _2POW(N)> m_data;
};

/**
 * @brief Transforme un qubit dans sa représentation sous forme de
 * vecteur en sa représentation classique.
 * Si le qubit contient deux cases non nuls, c'est une combinaison
 * linéaires de deux qubits que l'on peut écrire avec la forme classique,
 * c'est-à-dire avec des "0" et des "1".
 * @warning Cette fonction ne permet de faire la transformation que
 * pour au plus deux cases non nuls dans le qubits
 * @tparam N La taille du qubit
 * @param[in] qubit Le qubit à transformer
 * @return Dans chaque cases du tableau, on a le qubits
 * sous sa représentation classique que l'on modélise par
 * un tableau de booléen, et un complexe qui est la scalaire.
 */
template <std::size_t N>
CONSTEXPR std::array<std::pair<std::array<bool, N>, std::complex<double>>>, 2>
qubitToArray(Qubit<N> const &qubit);

/**
 * @brief Opérateur du produit entre un qubit et une matrice
 * @warning le produit n'est pas associatif
 * @tparam M La taille du qubit, la matrice est de taille 2^M
 * @param[in] lhs La matrice carré complexe de taille 2^M
 * @param[in] rhs Le M-qubit
 * @return Qubit<M> renvoie le qubit résultant du produit
 */
template <std::size_t N>

```

```

CONSTEXPR Qubit<N> operator*(
    CMatrix<_2POW(N)> const &lhs,
    Qubit<N> const &rhs);

/**
 * @brief Fonction d'affichage des qubit sous forme de
 * liste de nombre complexe
 *
 * @tparam M La taille du qubit
 * @param[out] out Le flux de destination
 * @param[in] qubit Le qubit à afficher
 * @return std::ostream& le flux de destination
 */
template <std::size_t N>
std::ostream &operator<<(std::ostream &out, Qubit<N> const &qubit);

#include "Qubit.tpp"

#endif

```

```

#include "Qubit.hpp"
#include <Matrix.hpp>
#include <algorithm>
#include <numeric>
#include <math_utility.hpp>
#include <Constexpr.hpp>
#include <Coord.hpp>

template <std::size_t N>
CONSTEXPR Qubit<N>::Qubit(
    std::array<std::complex<double>, _2POW(N)> &init_list)
    : m_data(std::move(init_list))
{
}

template <std::size_t N>
CONSTEXPR Qubit<N>::Qubit(std::array<bool, N> const &data) : m_data()
{
    auto sum{[] (std::array<bool, N> const &tab) -> int
        {
            int acc{0};
            for (std::size_t i{0}; i < std::size(tab); ++i)
                acc += _2POW(i) * tab[i];
            return acc;
        }};

    m_data[sum(data)] = 1;
}

```

```

template <std::size_t N>
CONSTEXPR Qubit<N> operator*(
    CMatrix<_2POW(N)> &lhs,
    Qubit<N> &rhs)
{
    std::array<std::complex<double>, _2POW(N)> tab{};
    for (std::size_t i{0}; i < lhs.numberLines(); i++)
    {
        for (std::size_t j{0}; j < lhs.numberColumns(); j++)
        {
            tab[i] += lhs(i, j) * rhs.m_data[j];
        }
    }

    return Qubit<N>(std::move(tab));
}

template <std::size_t N>
std::ostream &operator<<(
    std::ostream &out,
    Qubit<N> &qubit)
{
    for (std::size_t i{0}; i < _2POW(N); i++)
    {
        out << qubit.m_data[i].real() << "+" << qubit.m_data[i].imag()
    }
    return out;
}

```

```

template <std::size_t N>
CONSTEXPR std::array<
    std::pair<std::array<bool, N>,
                std::complex<double>>>,
    2>
qubitToArray(Qubit<N> const &qubit)
{
    std::array<
        std::pair<std::array<bool, N>,
                    std::complex<double>>>,
        2>
    tab{};

    bool b{true};
    std::size_t compteur{N};
    std::size_t pow = _2POW(N);
    for (std::size_t i{0}; i < pow; i++)
    {
        using namespace std::complex_literals;
        if (complex_equal(qubit.m_data[i], 0i))
        {
            if (i == pow - 1)
            {
                return tab;
            }
            else
            {
                if (b)
                {
                    while (tab[i].f

```

```

    }
    tab[0].first[co]
    tab[1].first[co]
    compteur = N;
}
else
{
    while (tab[1].f
    {

    }
    tab[1].first[co]
    compteur = N;
}

}

}
else
{
    if (b)
    {
        tab[0].second = qubit.m_data[i]
        b = false;
        if (i != pow - 1)
        {
            while (tab[1].f
            {

```

```

    }
    tab[1].first[co
    compteur = N;
}

}
else
{
    tab[1].second = qubit.m_data[i]
    return tab;
}

}
return tab;
}
}

```

```

#ifdef RANDOM_HPP
#define RANDOM_HPP

namespace rnd
{
    /**
     * @brief Renvoie un entier dans l'intervale [a, b]
     *
     * @param a, b Entier pour spécifier la plage
     * @return int Un entier entre a et b inclu
     */
    int randint(int a, int b);

    /**
     * @brief Renvoie un réel dans l'intervale [a, b)
     *
     * @param a, b Réel pour spécifier la plage
     * @return double Un réel entre a et b inclu
     */
    double randreal(double a, double b);
}

#endif

```



```

#include <random>
#include "Random.hpp"
#include <iostream>
namespace
{
    std::random_device rd;
}

namespace rnd
{
    int randint(int a, int b)
    {
        static std::uniform_int_distribution<> distrib(a, b);
        return distrib(rd);
    }

    double randreal(double a, double b)
    {
        static std::uniform_real_distribution<> distrib(a, b);
        return distrib(rd);
    }
}

```

```
#ifndef MATH_UTILITY_HPP
#define MATH_UTILITY_HPP

#define EPSILON 10e-3

#include <complex>
#include <Constexpr.hpp>

namespace
{
    CONSTEXPR bool double_equal(double x, double y);
    CONSTEXPR bool complex_equal(
        std::complex<double> const &z1,
        std::complex<double> const &z2);
}

#include "math_utility.hpp"
#endif
```

```

#include <cmath>
#include <complex>
#include <Constepr.hpp>
#include "math_utility.hpp"

namespace
{
    CONSTEXPR bool double_equal(double x, double y)
    {
        return std::abs(x - y) <= EPSILON;
    }

    CONSTEXPR bool complex_equal(
        std::complex<double> const &z1,
        std::complex<double> const &z2)
    {
        return double_equal(z1.real(), z2.real()) &&
            double_equal(z1.imag(), z2.imag());
    }
}

```

```

#ifdef CHECK_PATH_HPP
#define CHECK_PATH_HPP

#include <Coord.hpp>
#include <Board.hpp>
#include <Constexpr.hpp>

template <std::size_t N, std::size_t M>
CONSTEXPR static bool
check_path_straight(
    Board<N, M> const &board,
    Coord const &dpt,
    Coord const &arv) noexcept;

template <std::size_t N, std::size_t M>
CONSTEXPR static bool
check_path_diagonal(
    Board<N, M> const &board,
    Coord const &dpt,
    Coord const &arv) noexcept;

/**
 * @brief Vérifie si il y a une pièce entre deux cases sur une instance
 * du plateau pour un mouvement orthogonale
 *
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] board Le plateau
 * @param[in] dpt Les coordonnées de la case de départ
 * @param[in] arv Les coordonnées de la case d'arrivée

```

```

* @param[in] position L'instance du plateau
* @return true si le mouvement est possible
* @return false si le mouvement est bloqué,
* on si ce n'est pas un mouvement orthogonal
*/
template <std::size_t N, std::size_t M>
CONSTEXPR bool
check_path_straight_1_instance(
    Board<N, M> const &board,
    Coord const &dpt,
    Coord const &arv,
    std::size_t position,
    std::optional<Coord> position_other_piece_merge);

/**
 * @brief Vérifie si il y a une pièce entre deux cases sur une
 * instance du plateau pour un mouvement diagonale.
 *
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] board Le plateau
 * @param[in] dpt Les coordonnées de la case de départ
 * @param[in] arv Les coordonnées de la case d'arrivée
 * @param[in] position L'instance du plateau
 * @return true si le mouvement est possible
 * @return false si le mouvement est bloqué, on si ce n'est pas
 * un mouvement diagonale.
 */
template <std::size_t N, std::size_t M>
CONSTEXPR bool check_path_diagonal_1_instance(

```

```

Board<N, M> const &board,
Coord const &dpt,
Coord const &arv,
std::size_t position,
std::optional<Coord> position_other_piece_merge);

/**
 * @brief Vérifie si il y a une pièce entre deux cases
 * sur une instance du plateau pour un mouvement orthogonale ou diagonale.
 *
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonnes du plateau
 * @param[in] board Le plateau
 * @param[in] dpt Les coordonnées de la case de départ
 * @param[in] arv Les coordonnées de la case d'arrivée
 * @param[in] position L'instance du plateau
 * @return true si le mouvement est possible
 * @return false si le mouvement est bloqué, on si ce n'est pas
 * un mouvement orthogonal ou diagonale.
 */
template <std::size_t N, std::size_t M>
CONSTEXPR bool check_path_queen_1_instance(
    Board<N, M> const &board,
    Coord const &dpt,
    Coord const &arv,
    std::size_t position,
    std::optional<Coord> position_other_piece_merge);

#include "check_path.hpp"
#endif

```

```

#include <algorithm>
#include <Board.hpp>
#include <Coord.hpp>
#include <Constexpr.hpp>
#include "check_path.hpp"

template <std::size_t N, std::size_t M>
CONSTEXPR bool
check_path_straight(
    Board<N, M> const &board,
    Coord const &dpt,
    Coord const &arv) noexcept
{
    if (dpt.n == arv.n)
    {
        for (std::size_t i{std::min(dpt.m, arv.m) + 1};
             i < std::max(dpt.m, arv.m);
             i++)
        {
            if (board(dpt.n, i).get_type() != TypePiece::EM)
            {
                board.get_proba(Coord(dpt.n, i))
                return false;
            }
        }
        return true;
    }
    else if (dpt.m == arv.m)
    {
        for (std::size_t i{std::min(dpt.n, arv.n) + 1};

```

```

        i < std::max(dpt.n, arv.n);
        i++)
    {
        if (board(i, dpt.m).get_type() != TypePiece::EM
            board.get_proba(Coord(i, dpt.m))
        {
            return false;
        }
    }
    return true;
}
return false;
}

template <std::size_t N, std::size_t M>
CONSTEXPR bool
check_path_diagonal(
    Board<N, M> const &board,
    Coord const &dpt,
    Coord const &arv) noexcept
{
    std::size_t const max_lines{std::max(dpt.n, arv.n)};
    std::size_t const min_lines{std::min(dpt.n, arv.n)};
    std::size_t const max_columns{std::max(dpt.m, arv.m)};
    std::size_t const min_columns{std::min(dpt.m, arv.m)};

    std::size_t const dist{max_lines - min_lines};

    if (dist == max_columns - min_columns)
    {

```



```

        for (std::size_t i{1}; i < dist; i++)
        {
            if (board(min_lines + i, min_columns + i).get_t
            {
                return false;
            }
        }
        return true;
    }
    return false;
}

template <std::size_t N, std::size_t M>
constexpr bool
check_path_straight_1_instance(
    Board<N, M> const &board,
    Coord const &dpt,
    Coord const &arv,
    std::size_t position,
    std::optional<Coord> position_other_piece_merge)
{
    if (dpt.n == arv.n)
    {
        for (std::size_t i{std::min(dpt.m, arv.m) + 1};
             i < std::max(dpt.m, arv.m);
             i++)
        {
            if (board.m_board[position].first[board.offset(
            {
                if (position_other_piece_merge.

```

```

        {
            if (!(board.off
        {
        }
    }
    else
    {
        return false;
    }
}

return true;
}

}

else if (dpt.m == arv.m)
{
    for (std::size_t i{std::min(dpt.n, arv.n) + 1};
        i < std::max(dpt.n, arv.n);
        i++)
    {
        if (board.m_board[position].fir
        {
            if (position_ot
            {

```

```

    }
    else
    {

    }

    }

    }
    return true;
}
return false;
}

template <std::size_t N, std::size_t M>
CONSTEXPR bool
check_path_diagonal_1_instance(
    Board<N, M> const &board,
    Coord const &dpt,
    Coord const &arv,
    std::size_t position,
    std::optional<Coord> position_other_piece_merge)
{
    std::size_t const max_lines{std::max(dpt.n, arv.n)};
    std::size_t const min_lines{std::min(dpt.n, arv.n)};
    std::size_t const max_columns{std::max(dpt.m, arv.m)};
    std::size_t const min_columns{std::min(dpt.m, arv.m)};

    std::size_t const dist{max_lines - min_lines};

    if (dist == max_columns - min_columns)

```

```

{
    for (std::size_t i{1}; i < dist; i++)
    {
        if (board.m_board[position]

                                if (position_ot
                                {

                                }
                                else
                                {

                                }

        }

    }
    return true;
}
return false;
}

template <std::size_t N, std::size_t M>
CONSTEXPR bool check_path_queen_i_instance(
    Board<N, M> const &board,

```

```

Coord const &dpt,
Coord const &arv,
std::size_t position,
std::optional<Coord> position_other_piece_merge)
{
    return check_path_straight_1_instance<N, M>(board, dpt, arv, po
        check_path_diagonal_1_instance<N, M>(b
}

```

```

#ifdef UNITARY_HPP
#define UNITARY_HPP
#include <CMatrix.hpp>
#include <numbers>
#include <complex>
#include <Constexpr.hpp>

using namespace std::complex_literals;
using namespace std::numbers;

constexpr inline
CMatrix<4> MATRIX_ISWAP {
    {
        {1, 0, 0, 0},
        {0, 0, 1i, 0},
        {0, 1i, 0, 0},
        {0, 0, 0, 1}
    }
};

constexpr inline
CMatrix<4> MATRIX_SQRT_ISWAP {
    {
        {1, 0, 0, 0},
        {0, 1./sqrt2, 1i/sqrt2, 0},
        {0, 1i/sqrt2, 1./sqrt2, 0},
        {0, 0, 0, 1}
    }
};

```

```
constexpr inline
CMatrix<4> MATRIX_JUMP { MATRIX_ISWAP };

constexpr inline
CMatrix<8> MATRIX_SLIDE {
    {
        {1, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 1i, 0, 0, 0, 0, 0},
        {0, 1i, 0, 0, 0, 0, 0, 0},
        {0, 0, 0, 1, 0, 0, 0, 0},
        {0, 0, 0, 0, 1, 0, 0, 0},
        {0, 0, 0, 0, 0, 1, 0, 0},
        {0, 0, 0, 0, 0, 0, 1, 0},
        {0, 0, 0, 0, 0, 0, 0, 1}
    }
};

constexpr inline
CMatrix<8> MATRIX_ISWAP_8 {
    {
        {1, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0, 1i, 0, 0},
        {0, 0, 1, 0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0, 0, 0, 1i},
        {0, 1i, 0, 0, 0, 0, 0, 0}, // erreur dans le papier?
        {0, 0, 0, 0, 0, 1, 0, 0},
        {0, 0, 0, 1i, 0, 0, 0, 0},
        {0, 0, 0, 0, 0, 0, 0, 1}
    }
}
```

```

};

constexpr inline
CMatrix<8> MATRIX_SQRT_ISWAP_8 {
    {
        {1,          0,          0, 0, 0,          0,          0, 0},
        {0, 1./sqrt2, 1i/sqrt2, 0, 0,          0,          0, 0},
        {0, 1i/sqrt2, 1./sqrt2, 0, 0,          0,          0, 0},
        {0,          0,          0, 1, 0,          0,          0, 0},
        {0,          0,          0, 0, 1,          0,          0, 0},
        {0,          0,          0, 0, 0, 1./sqrt2, 1i/sqrt2, 0},
        {0,          0,          0, 0, 0, 1i/sqrt2, 1./sqrt2, 0},
        {0,          0,          0, 0, 0,          0,          0, 1}
    }
};

constexpr inline
CMatrix<8> MATRIX_SPLIT {
    {
        {1,          0,          0, 0, 0,          0,          0, 0},
        {0,          0,          0, 0, 1i,          0,          0, 0},
        {0, 1i/sqrt2, 1./sqrt2, 0, 0,          0,          0, 0},
        {0,          0,          0, 0, 0, -1./sqrt2, 1i/sqrt2, 0},
        {0, 1i/sqrt2, -1./sqrt2, 0, 0,          0,          0, 0},
        {0,          0,          0, 0, 0, 1i/sqrt2, -1/sqrt2, 0},
        {0,          0,          0, 1i, 0,          0,          0, 0},
        {0,          0,          0, 0, 0,          0,          0, 1}
    }
};

```



```
constexpr inline
CMatrix<8> MATRIX_MERGE {
    {
        {1, 0, 0, 0, 0, 0, 0, 0},
        {0, 0, -1i/sqrt2, 0, -1i/sqrt2, 0, 0, 0},
        {0, 0, 1/sqrt2, 0, -1/sqrt2, 0, 0, 0},
        {0, 0, 0, 0, 0, 0, -1i, 0},
        {0, -1i, 0, 0, 0, 0, 0, 0},
        {0, 0, 0, -1/sqrt2, 0, -1i/sqrt2, 0, 0},
        {0, 0, 0, -1i/sqrt2, 0, -1/sqrt2, 0, 0},
        {0, 0, 0, 0, 0, 0, 0, 1}
    }
};

constexpr inline
CMatrix<32> MATRIX_SPLIT_SLIDE {
    CMatrix<16>{
        MATRIX_SPLIT, CMatrix<8>{},
        CMatrix<8>{}, MATRIX_ISWAP_8
    },
    CMatrix<16>{},
    CMatrix<16>{

};

CONSTEXPR inline
CMatrix<32> MATRIX_MERGE_SLIDE {
    CMatrix<16>{
```

```

MATRIX_MERGE, CMatrix<8>{},
CMatrix<8>{}, conj(CMatrix<2>::identity()).tensoriel_product(MAT
},
CMatrix<16>{},
CMatrix<16>{},
CMatrix<16>{
conj(MATRIX_ISWAP_8.transposed()), CMatrix<8>{},
CMatrix<8>{}, CMatrix<8>::identity()
}
};

#endif

```

```

#ifdef CMATRIX_HPP
#define CMATRIX_HPP
#include <Matrix.hpp>
#include <complex>
#include <Constexpr.hpp>
#include <cstddef>

/**
 * @brief Objet représentant les matrices carrées complexe
 * de dimension N
 *
 * @tparam N La dimension de la matrice
 */
template <std::size_t N>
using CMatrix = Matrix<std::complex<double>, N>;

namespace
{
    template <std::size_t N>
    CONSTEXPR CMatrix<N> conj(CMatrix<N> const & matrix)
    {
        CMatrix<N> matrix_conj{};
        for(std::size_t i = 0; i<N; i++)
        {
            for(std::size_t j = 0; j<N; j++)
            {
                matrix_conj(i, j) = std::conj(matrix(i, j));
            }
        }
        return matrix_conj;
    }
}

```

```
    }  
}  
  
/**  
 * @brief Réalise l'opération  $2^n$   
 * @warning Est valable uniquement sur les types entiers  
 */  
#define _2POW(n) (1 << (n))  
  
#endif
```

```

#ifndef COMPLEX_PRINTER_HPP
#define COMPLEX_PRINTER_HPP

#include <complex>
#include <concepts>
#include <string>

namespace std
{
    /**
     * @brief convertit un nombre complexe en chaîne de caractère
     */
    using namespace std::literals;
    template <std::floating_point T>
    std::string to_string(std::complex<T> value)
    {
        return std::string{std::to_string(std::real(value))}

    }

    /**
     * @brief Implémente l'opérateur de flux sortant
     * sur les nombres complexes
     *
     * @tparam T le type de représentation floatante du complexe
     * @param os Le flux de sorti
     * @param value Le nombre complexe à transférer
     * @return std::ostream& retourne le flux passé en paramètre
     */

```

```
template <std::floating_point T>
std::ostream& operator<<(std::ostream& os, std::complex<T> value)
{
    os << std::real(value) << ((std::imag(value) >= 0) ? '+' : '\0')
    return os;
}

#endif
```

```
#ifndef CONSTEXPR_HPP
#define CONSTEXPR_HPP

/**
 * @brief Utilisé pour utiliser ou non constexpr
 */
#define CONSTEXPR constexpr

#endif
```

```

#ifndef COMPUTER_PLAYER_HPP
#define COMPUTER_PLAYER_HPP

#include <cstdint>
#include <Board.hpp>
#include <Constexpr.hpp>

namespace computer
{
    template <std::size_t N, std::size_t M>
    CONSTEXPR Move
    get_best_move(
        Board<N, M> const &board,
        int profondeur);
}

#include "ComputerPlayer.hpp"
#endif

```



```

#include <Constexpr.hpp>
#include <cstdint>
#include <forward_list>
#include <functional>
#include <limits>
#include <tuple>
#include <algorithm>
#include <thread>
#include <concepts>
#include <unordered_map>
#include <Board.hpp>
#include <TypePiece.hpp>
#include <Color.hpp>
#include <Coord.hpp>
#include <Move.hpp>
#include <mutex>
#include <Random.hpp>

#define WIN_VALUE 15.

namespace computer
{
    namespace __utility
    {
        /**
         * @brief Renvoi la valeur d'une pièce
         *
         * @param piece Le type de la pièce
         * @return La valeur de la pièce
         */
    }
}

```

```

CONSTEXPR int value_piece(TypePiece piece)
{
    switch (piece)
    {
        case TypePiece::PAWN:
            return 1;
        case TypePiece::KNIGHT:
            return 3;
        case TypePiece::BISHOP:
            return 3;
        case TypePiece::ROOK:
            return 5;
        case TypePiece::QUEEN:
            return 9;
        case TypePiece::KING:
            return 5;
        case TypePiece::EMPTY:
        default:
            return 0;
    }
}

/**
 * @brief Renvoi le signe d'un joueur
 *
 * @param color La couleur du joueur
 * @return 1 si c'est le joueur blanc
 * @return -1 si c'est le joueur noir
 */
CONSTEXPR int sign_color(Color color)

```

```

{
    return (color == Color::WHITE) ? 1 : -1;
}

/**
 * @brief Teste si un plateau est gagnant et indique le gagnant
 *
 * @tparam N Le nombre de ligne du plateau
 * @tparam M Le nombre de colonne du plateau
 * @param[in] board Le plateau à tester
 * @param[out] winner_color prend la couleur du joueur gagnant.
 * Contenu non garanti si il n'y a pas de gagnant
 * @return true si il y a un gagnant
 * @return false sinon
 */
template <std::size_t N, std::size_t M>
CONSTEXPR bool winner(Board<N, M> const& board, Color &winner_color)
{
    bool found_W_king{false}, found_B_king{false};
    for (std::size_t i{0}; i < board.numberLines(); i++)
    {
        for (std::size_t j{0}; j < board.numberColumns(); j++)
        {
            auto piece = board(i, j);
            if (piece.get_type() == TypePiece::KING)
            {
                if (piece.get_color() == Color::WHITE)
                {
                    found_W_king = true;
                }
            }
        }
    }
}

```

```

        else
        {
            found_B_king = true;
        }
        if (found_W_king == found_B_king)
        {
            return false;
        }
    }
}

if (found_B_king)
{
    winner_color = Color::BLACK;
}
else
{
    winner_color = Color::WHITE;
}
return true;
}

/**
 * @brief Renvoie la fonction permettant de calculer le meilleur
 * score du joueur
 *
 * @param[in] board Le plateau de jeu
 * @param[in] x Le premier score
 * @param[in] y Le second score
 * @return true si le second score est plus

```

```

    * interessant pour le joueur
    * return false sinon
    */
CONSTEXPR bool
get_player_calc_best_score(Color color,

{
    if (double_equal(x, y))
    {
        return static_cast<bool>(rnd::randint(0, 1));
    }
    if (color == Color::WHITE)
    {
        return (y > x) ? true : false;
    }
    return (y < x) ? true : false;
}

template <std::size_t N, std::size_t M>
bool check_alpha_beta(
    Board<N, M> const &board,
    std::forward_list<double> const &best_all_round_score,
    double &best_score,
    double score)
{
    if (get_player_calc_best_score(
        board.get_current_player(),
        best_score, score))
    {

```

```

best_score = score;
Color current{board.get_current_player()};
for (double const alpha : best_all_round_score)
{
    if (get_player_calc_best_score(
                                current,
                                alpha,
                                best_score))
    {
        return true;
    }
    if (current == Color::WHITE)
    {
        current = Color::BLACK;
    }
    else
    {
        current = Color::WHITE;
    }
}
return false;
}

template <std::size_t N, std::size_t M>
constexpr double evalCase(std::size_t i, std::size_t j) noexcept
{
    if constexpr (N % 2 == 0)
    {
        if constexpr (M % 2 == 0)

```

```

        {
            return 2*std::min(std::min(static_cast<double>(i), static_cast<double>(j)),
                               std::min(static_cast<double>(j), static_cast<double>(M - i - j)));
        }
        else
        {
            return 2*std::min(std::min(static_cast<double>(i), static_cast<double>(j)),
                               std::min(static_cast<double>(j), static_cast<double>(M - i - j)));
        }
    }
    else
    {
        if constexpr (M % 2 == 0)
        {
            return 2*std::min(std::min(static_cast<double>(i), static_cast<double>(j)),
                               std::min(static_cast<double>(j), static_cast<double>(M - i - j)));
        }
        else
        {
            return 2*std::min(std::min(static_cast<double>(i), static_cast<double>(j)),
                               std::min(static_cast<double>(j), static_cast<double>(M - i - j)));
        }
    }
}

/**
 * @brief Renvoie une évaluation du plateau
 * @details L'évaluation du plateau est calculé
 * en sommant toutes les pièces d'un joueur
 * coéfficienté par leurs probabilités et en soustrayant

```

```

* avec ce meme calcul les piéces de l'autre joueur
* @tparam N Le nombre de ligne du plateau
* @tparam M Le nombre de colonne du plateau
* @param board Le plateau de jeu
* @return double L'évaluation du plateau
*/
template <std::size_t N, std::size_t M>
CONSTEXPR double heuristic(Board<N, M> const &board) noexcept
{
    Color win;
    if (__utility::winner(board, win))
    {
        return __utility::sign_color(win) * WIN_VALUE;
    }
    double h{0};
    for (std::size_t i{0}; i < board.numberLines(); i++)
    {
        for (std::size_t j{0}; j < board.numberColumns(); j++)
        {
            auto p = board(i, j);
            if (p.get_type() != TypePiece::EMPTY)
            {
                h += sign_color(p.get_color()) *
                    (__utility::value_piece(p.get_type(),
                        evalCase<N, M>(i, j)) *
                    board.get_proba(Coord(i, j)));
            }
        }
    }
    return h;
}

```



```

}

template <std::size_t N, std::size_t M>
CONSTEXPR double rec_get_best_move(
    Board<N, M> const &board,
    std::forward_list<double> &best_score_alpha_beta,
    int profondeur);

template <std::size_t N, std::size_t M>
CONSTEXPR double deterministic_eval_move(
    Board<N, M> const &board,
    std::forward_list<double> &best_score_alpha_beta,
    Move const &move,
    int profondeur)
{
    double proba_move{board.get_proba_move(move)};
    Board<N, M> board_cpy1{board};
    if (double_equal(proba_move, 1.))
    {
        board_cpy1.move(move, true);
        board_cpy1.change_player();
        return rec_get_best_move(board_cpy1, best_score_alpha_beta, pro
    }
    else if (double_equal(proba_move, 0.))
    {
        board_cpy1.move(move, false);
        board_cpy1.change_player();
        return rec_get_best_move(board_cpy1, best_score_alpha_beta, pro
    }
    else

```

```

{
    Board<N, M> board_cpy2{board};
    board_cpy1.move(move, true);
    board_cpy2.move(move, false);
    board_cpy1.change_player();
    board_cpy2.change_player();
    double eval1{rec_get_best_move(board_cpy1, best_score_alpha_beta,
    double eval2{rec_get_best_move(board_cpy2, best_score_alpha_beta,

    return proba_move * eval1 + (1. - proba_move) * eval2;
}
}

template <std::size_t N, std::size_t M>
constexpr double rec_get_best_move(
    Board<N, M> const &board,
    std::forward_list<double> &best_score_alpha_beta,
    int profondeur)
{
    Color win;
    if (profondeur <= 0)
    {
        return heuristic(board);
    }
    else if (winner(board, win))
    {
        return __utility::sign_color(win) * WIN_VALUE;
    }
    else
    {

```

```

double best_score{-1 * sign_color(board.get_current_player()) *

board.all_move(
    [&board,
     &best_score_alpha_beta,
     profondeur,
     &best_score](Move const &m) mutable -> bool
    {
        best_score_alpha_beta
            .push_front(best_score)
        double score;
        try
        {
            score = deterministic_eval_move(
                board,
                best_score_alpha_beta,
                m,
                profondeur);
        }
        catch (std::runtime_error const &e)
        {
            return false;
        }
        best_score_alpha_beta.pop_front();
        return check_alpha_beta(
            board,
            best_score_alpha_beta,
            best_score,
            score);
    }

```

```

        });
        return best_score;
    }
}

template <std::size_t N, std::size_t M>
struct Param_get_best_move
{
    Board<N, M> board;
    std::forward_list<Move> const &move;
    double &best_eval;
    Move &best_move;
    int profondeur;
};

template <std::size_t N, std::size_t M>
CONSTEXPR void
th_get_best_move(Param_get_best_move<N, M> &&param)
{
    param.best_eval = -1 * sign_color(param.board.get_current_player()) *

    for (Move const &move : param.move)
    {
        double score;
        Board<N, M> board_cpy = param.board;
        if (move.type == TypeMove::PROMOTE)
        {
            board_cpy.move_promotion(move);
        }
        else

```

```

        {
            board_cpy.move(move);
        }
        board_cpy.change_player();
        std::forward_list<double> list_best_score{param.best_eval};
        score = rec_get_best_move(board_cpy,

                                if (get_player_calc_best_score(
                                    param.board.get_current_player(
                                        param.best_eval, score))

                                {
                                    param.best_eval = score;
                                    param.best_move = move;
                                }
                            }
    }

template <std::size_t N, std::size_t M>
CONSTEXPR Move get_best_move(
    Board<N, M> const &board,
    int profondeur)
{
    unsigned int number_thread{std::thread::hardware_concurrency()};
    std::vector<std::forward_list<Move>> list_move(number_thread);
    std::size_t last_th_view{0};

    board.all_move(

```

```

        [&list_move,
         &last_th_view,
         number_thread](Move const &m) mutable -> bool
        {
            list_move[last_th_view].push_front(std::move(m));
            last_th_view = (last_th_view + 1) % number_thread;
            return false;
        });

    std::vector<std::thread> threads(number_thread);

    std::vector<double> best_eval(number_thread);

    std::vector<Move> best_move(number_thread);

    for (std::size_t i{0}; i < number_thread; i++)
    {

        __utility::Param_get_best_move param{board,

        // param.board = board;
        threads[i] = std::thread(__utility::th_get_best_move<N, M>, param);
    }
    double better{-1 * __utility::sign_color(board.get_current_player()) *
                  std::numeric_limits<double>::max()};

    Move better_move;
    for (std::size_t i{0}; i < number_thread; i++)

```

```

    {
        threads[i].join();
        if (__utility::get_player_calc_best_score(
                                                    board.get_current_player(),
                                                    better,
                                                    best_eval[i]))
        {
            better = best_eval[i];
            better_move = best_move[i];
        }
    }
    return better_move;
}

```

```
#ifndef CONSOLE_INTERFACE_HPP
#define CONSOLE_INTERFACE_HPP

#include <ostream>
#include <Board.hpp>

template <std::size_t N, std::size_t M>
std::ostream& operator<<(std::ostream& os, Board<N, M> const& board);

#include "ConsoleInterface.hpp"
#endif
```



```

#include <iostream>
#include <Board.hpp>

const char *getUnicodeChar(TypePiece piece, Color color)
{
    using enum TypePiece;
    if (color == Color::WHITE)
    {
        switch (piece)
        {
            case KING:
                return "K";
            case QUEEN:
                return "Q";
            case ROOK:
                return "R";
            case BISHOP:
                return "B";
            case KNIGHT:
                return "N";
            case PAWN:
                return "P";
            case EMPTY:
            default:
                return " ";
        }
    }
    else
    {
        switch (piece)

```

```

        {
            case KING:
                return "k";
            case QUEEN:
                return "q";
            case ROOK:
                return "r";
            case BISHOP:
                return "b";
            case KNIGHT:
                return "n";
            case PAWN:
                return "p";
            case EMPTY:
            default:
                return " ";
        }
    }

}

template <std::size_t N, std::size_t M>
std::ostream &operator<<(std::ostream &os, Board<N, M> const &board)
{
    for (std::size_t i{0}; i < M; i++)
    {
        os << "|-----";
    }
    os << "\n";
    for (std::size_t i{0}; i < N; i++)
    {

```

```

for (std::size_t j{0}; j < M; j++)
{
    TypePiece piece{board(i, j).get_type()};
    Color color{board(i, j).get_color()};
    os << "| " << getUnicodeChar(piece, color) << " ";
}
os << "|\n";
for (std::size_t j{0}; j < M; j++)
{
    if (board(i, j).get_type() != TypePiece::EMPTY)
    {
        double proba{board.get_proba(Coord(i, j))};
        int p{static_cast<int>(std::round(100. * proba))};
        if (p != 100)
        {
            os << "| ";
            if (p < 10){
                os << '0';
            }
            os << p << "% ";
        }
        else
        {
            os << "| ";
        }
    }
    else
    {
        os << "| ";
    }
}

```

```

    }
    os << "\\n";
    for (std::size_t j{0}; j < M; j++)
    {
        os << "|-----";
    }
    os << "\\n";
}
(void)board;
return os;
}

```